

AKADEMIA NAUK STOSOWANYCH W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych

Bazy danych

studia niestacjonarne
semestr letni 2025/2026

Cel i zakres projektu:

- 1) Zaprojektowanie architektury oraz logiki biznesowej dla bezpiecznego systemu cyfrowego zarządzania kadrami, czasem pracy i obiegiem dokumentów (WorkFlow).
- 2) Analiza wymagań funkcjonalnych i нефункцийных ze szczególnym uwzględnieniem mechanizmów bezpieczeństwa (ochrona danych osobowych, kontrola dostępu oparta na rolach – RBAC oraz bezpieczny przepływ wrażliwych informacji finansowych).

Nazwisko i Imię:

Curzydło Rafał

Nr grupy:

Data oddania: 11.04.2026

Ocena:

Nowy Sącz 2026

Spis treści

1. Tytuł	5
1.1. Definicja i zakres przedmiotowy	5
1.2. Charakterystyka rozwiązania	6
1.3. Zakres dokumentacji	6
1.4. Znaczenie dla dalszej części dokumentu	7
2. Nazwa kodowa	7
2.1. Znaczenie nazwy	7
2.2. Zastosowanie nazwy w strukturze projektu	8
2.3. Uzasadnienie wyboru	8
2.4. Rola nazwy w dalszej dokumentacji	9
3. Cel projektu	9
3.1. Uporządkowanie danych kadrowych	9
3.2. Obsługa struktury organizacyjnej	10
3.3. Rejestracja i weryfikacja czasu pracy	10
3.4. Obsługa dokumentacji pracowniczej	10
3.5. Kontrola ważności certyfikatów i badań	10
3.6. Rejestracja nieobecności	11
3.7. Przygotowanie danych do rozliczeń	11
3.8. Kontrola dostępu i role użytkowników	11
3.9. Cel techniczny projektu	12
3.10. Zakładany rezultat	12
4. Analiza wymagań	12
4.1. Podejście do analizy	13
4.2. Zakres wymagań	13
4.3. Założenia techniczne	14
4.4. Podział wymagań	15
4.5. Wymagania niefunkcyjne	15
4.6. Wymagania funkcjonalne	19
4.7. Wymagania integracyjne	27
5. Diagram przypadków użycia	33
5.1. Diagram ogólny systemu	34
5.2. Moduł kadrowy	35

5.3. Czas pracy, RCP i rozliczenia	36
5.4. Administracja i struktura organizacyjna	37
5.5. Aktorzy systemu	37
5.6. Zakres przypadków użycia	38
5.7. Powiązanie ról z funkcjami	39
5.8. Znaczenie diagramu dla projektu	39
6. Scenariusze	40
6.1. SC-01: Logowanie użytkownika	40
6.2. SC-02: Zarządzanie profilem pracownika	41
6.3. SC-03: Rejestracja czasu pracy przez czytnik RCP	42
6.4. SC-04: Weryfikacja wpisów czasu pracy	43
6.5. SC-05: Dodanie umowy pracownika	44
6.6. SC-06: Monitorowanie ważności certyfikatów	45
6.7. SC-07: Dodanie nieobecności pracownika	46
6.8. SC-08: Dodanie dokumentu pracownika	46
6.9. SC-09: Przygotowanie eksportu rozliczeniowego	47
6.10. SC-10: Uruchomienie danych demonstracyjnych	48
7. Makiety i interfejs użytkownika	49
7.1. Założenia projektowe interfejsu	49
7.2. Pulpit główny	50
7.3. Profil pracownika	52
7.4. Widoki tabelaryczne	54
7.5. Formularze	54
7.6. Tryb jasny i ciemny	55
7.7. Znaczenie makiet dla implementacji	55
8. Model danych i diagram ERD	56
8.1. Założenia modelu danych	57
8.2. Podział modelu na obszary	57
8.3. Słownik głównych encji	58
8.4. Struktura organizacyjna	60
8.5. Rejestracja czasu pracy	60
8.6. Kadry i dokumentacja	61
8.7. Rozliczenia	62
8.8. Audit log	62

8.9. Typy wyliczeniowe	62
8.10. Implementacja fizyczna bazy danych	63
8.11. Strategia dostępu do danych	64
8.12. Znaczenie modelu danych dla systemu	64
9. Wykorzystanie narzędzi AI w procesie realizacji projektu	65
9.1. Zakres wykorzystania narzędzi AI	65
9.2. Wsparcie podczas implementacji	66
9.3. Wsparcie przy konfiguracji środowiska	66
9.4. Wsparcie przy danych demonstracyjnych	67
9.5. Wsparcie przy dokumentacji	68
9.6. Weryfikacja odpowiedzi generowanych przez AI	68
9.7. Porównanie wykorzystanych narzędzi	69
9.8. Wpływ AI na przebieg pracy	69
9.9. Podsumowanie	70
10. Stos technologiczny	70
10.1. Warstwa prezentacji	71
10.2. Warstwa logiki biznesowej	72
10.3. Warstwa danych	73
10.4. Magazyn plików	73
10.5. Warstwa infrastruktury	74
10.6. Seedowanie danych	75
10.7. Komunikacja pomiędzy usługami	75
10.8. Uzasadnienie wyboru technologii	75
Literatura	77
Spis rysunków	79
Spis tabel	80
Spis listingów	81

1. Tytuł

Oficjalnym tytułem projektowanego rozwiązania jest: **WorkFlow – system wspierający zarządzanie pracownikami, czasem pracy i dokumentacją kadrową**. Tytuł ten odzwierciedla główny zakres aplikacji, który obejmuje ewidencję pracowników, przypisywanie ich do zakładów i projektów, rejestrację czasu pracy, obsługę dokumentów kadrowych, kontrolę certyfikatów oraz przygotowanie danych do rozliczeń.

Nazwa **WorkFlow** została przyjęta jako nazwa techniczna i projektowa systemu. Wskazuje ona na uporządkowany przepływ informacji pomiędzy poszczególnymi modułami aplikacji oraz rolami użytkowników. W projekcie istotne jest nie tylko samo przechowywanie danych, lecz także zachowanie zależności pomiędzy pracownikiem, zakładem, projektem, czasem pracy, dokumentacją oraz procesem rozliczeniowym.

1.1. Definicja i zakres przedmiotowy

System **WorkFlow** jest aplikacją webową typu full-stack, zaprojektowaną w celu wsparcia podstawowych procesów kadrowo-organizacyjnych w przedsiębiorstwie. Rozwiązanie składa się z warstwy frontendowej, warstwy backendowej, relacyjnej bazy danych, magazynu plików oraz infrastruktury uruchomieniowej opartej o kontenery.

Zakres przedmiotowy projektu obejmuje kilka powiązanych ze sobą obszarów:

- **Zarządzanie pracownikami** – prowadzenie centralnej ewidencji pracowników, ich danych podstawowych, statusu aktywności, roli systemowej oraz przypisania do zakładu.
- **Obsługa zakładów i projektów** – organizowanie danych według zakładów oraz przypisywanie pracowników do projektów, które reprezentują miejsca lub obszary wykonywania pracy.
- **Rejestracja czasu pracy** – przyjmowanie zdarzeń wejścia i wyjścia z czytników RCP oraz tworzenie na ich podstawie wpisów czasu pracy.
- **Dokumentacja kadrowa** – przechowywanie informacji o umowach, dokumentach, certyfikatach, badaniach, szkoleniach oraz nieobecnościach pracowników.
- **Rozliczenia** – przygotowanie uporządkowanych danych na potrzeby rozliczeń kadrowo-płacowych, w szczególności na podstawie zatwierdzonych wpisów czasu pracy.

W takim ujęciu system nie jest pełnym systemem ERP ani kompletnym narzędziem księgowym. Jego zadaniem jest wsparcie wybranego zakresu procesów związanych z obsługą pracowników i czasu pracy oraz przygotowanie danych, które mogą zostać wykorzystane przez osoby odpowiedzialne za kadry i rozliczenia.

1.2. Charakterystyka rozwiązania

Projekt ma charakter praktyczny i został przygotowany jako działająca aplikacja, możliwa do uruchomienia w środowisku lokalnym lub serwerowym. Przyjęta architektura rozdziela odpowiedzialności pomiędzy kilka warstw systemu:

- **frontend** odpowiada za interfejs użytkownika i komunikację z użytkownikiem końcowym,
- **backend** realizuje logikę biznesową, autoryzację, walidację danych oraz komunikację z bazą danych,
- **baza danych PostgreSQL** przechowuje dane relacyjne systemu,
- **MinIO** służy jako magazyn plików i dokumentów,
- **Nginx** pełni funkcję reverse proxy dla aplikacji.

Istotnym elementem projektu jest również podział użytkowników według ról. System przewiduje role takie jak administrator, pracownik HR, księgowość, brygadzysta, pracownik oraz rola biurowa. Dzięki temu dostęp do funkcji i danych może być ograniczany zgodnie z zakresem odpowiedzialności danej osoby.

1.3. Zakres dokumentacji

Niniejsza dokumentacja przedstawia zarówno założenia projektowe, jak i techniczne aspekty wykonania systemu. Obejmuje ona opis celu projektu, analizę wymagań, przypadki użycia, scenariusze działania, makiety interfejsu, model danych oraz zastosowany stos technologiczny.

Szczególną uwagę poświęcono modelowi danych, ponieważ to on stanowi podstawę działania większości modułów systemu. Relacje pomiędzy pracownikami, zakładami, projektami, czasem pracy, dokumentami, certyfikatami i rozliczeniami zostały przedstawione w postaci diagramu ERD oraz opisane w dalszej części dokumentu.

1.4. Znaczenie dla dalszej części dokumentu

Tak sformułowany tytuł i zakres projektu stanowią punkt wyjścia dla dalszej analizy. W kolejnych rozdziałach opisano nazwę kodową systemu, cele jego realizacji, wymagania funkcjonalne i нефункционалне, scenariusze użycia oraz strukturę danych.

Dokumentacja ma na celu przedstawienie aktualnego stanu projektu w sposób uporządkowany i możliwy do zweryfikowania. Z tego względu opis skupia się na funkcjach rzeczywiście przewidzianych lub zaimplementowanych w systemie, bez rozszerzania zakresu o moduły, które nie są częścią obecnej wersji aplikacji.

2. Nazwa kodowa

Na potrzeby projektu przyjęto nazwę kodową **WorkFlow**. Nazwa ta jest wykorzystywana w dokumentacji, repozytorium kodu źródłowego, konfiguracji środowiska uruchomieniowego oraz w interfejsie użytkownika aplikacji.

- **Nazwa projektu:** WorkFlow
- **Typ rozwiązania:** aplikacja webowa typu full-stack
- **Główna domena:** zarządzanie pracownikami, czasem pracy, dokumentacją kadrową i danymi rozliczeniowymi

2.1. Znaczenie nazwy

Nazwa **WorkFlow** odnosi się do przepływu pracy oraz przepływu informacji pomiędzy modułami systemu. W projekcie dane nie są traktowane jako niezależne rekordy, lecz jako elementy powiązanego procesu. Przykładowo pracownik jest przypisany do zakładu, może zostać przydzielony do projektu, posiada umowę, certyfikaty, dokumenty, nieobecności oraz wpisy czasu pracy.

Z tego względu nazwa systemu dobrze oddaje jego podstawowe założenie: uporządkowanie informacji i operacji wykonywanych przez różne role użytkowników. System wspiera przepływ danych pomiędzy częścią kadrową, organizacyjną, ewidencją czasu pracy oraz rozliczeniami.

2.2. Zastosowanie nazwy w strukturze projektu

Nazwa `WorkFlow` została zastosowana jako wspólny identyfikator całego rozwiązania. W praktyce występuje ona w kilku obszarach:

- jako nazwa aplikacji widoczna dla użytkownika,
- jako nazwa repozytorium projektu,
- jako nazwa środowiska uruchomieniowego w konfiguracji Docker Compose,
- jako element nazw kontenerów i usług technicznych,
- jako określenie systemu w dokumentacji projektowej.

Takie podejście pozwala zachować spójność pomiędzy warstwą techniczną, dokumentacją oraz sposobem prezentowania projektu użytkownikowi końcowemu.

2.3. Uzasadnienie wyboru

Wybór nazwy `WorkFlow` wynika z charakteru aplikacji. System nie ogranicza się do jednej funkcji, takiej jak samo przechowywanie danych pracowników lub sama ewidencja czasu pracy. Jego zadaniem jest połączenie kilku powiązanych procesów w jedną aplikację.

Najważniejszymi procesami objętymi projektem są:

- utworzenie i utrzymanie profilu pracownika,
- przypisanie pracownika do zakładu i projektu,
- rejestracja zdarzeń czasu pracy,
- utworzenie wpisów czasu pracy na podstawie zdarzeń z czytników RCP,
- obsługa umów i dokumentów kadrowych,
- kontrola certyfikatów, badań i szkoleń,
- rejestrowanie nieobecności,
- przygotowanie danych do rozliczeń.

Nazwa jest krótka, czytelna i neutralna technologicznie. Nie wskazuje na jedną konkretną funkcję, lecz na ogólną ideę organizacji pracy i danych w systemie.

2.4. Rola nazwy w dalszej dokumentacji

W dalszej części dokumentacji nazwa *WorkFlow* będzie stosowana jako określenie całego systemu. Obejmuje ona zarówno część frontendową, backendową, bazę danych, magazyn plików, jak i konfigurację uruchomieniową.

Dzięki temu możliwe jest jednoznaczne odróżnienie systemu jako całości od jego poszczególnych modułów, takich jak moduł pracowników, projektów, czasu pracy, certyfikatów, nieobecności, dokumentów czy rozliczeń.

3. Cel projektu

Celem projektu *WorkFlow* jest zaprojektowanie oraz wykonanie aplikacji webowej wspierającej zarządzanie pracownikami, czasem pracy, dokumentacją kadrową oraz danymi wykorzystywanymi w procesie rozliczeń. System ma uporządkować informacje, które w wielu organizacjach są przechowywane w rozproszony sposób, na przykład w arkuszach kalkulacyjnych, dokumentach papierowych, wiadomościach e-mail lub niezależnych plikach.

Projekt zakłada utworzenie jednego środowiska, w którym możliwe jest prowadzenie ewidencji pracowników, zakładów, projektów, umów, certyfikatów, dokumentów, nieobecności oraz wpisów czasu pracy. Dane te są ze sobą powiązane, dlatego ważnym celem systemu jest zachowanie spójności pomiędzy poszczególnymi obszarami aplikacji.

3.1. Uporządkowanie danych kadrowych

Jednym z podstawowych celów projektu jest stworzenie centralnej ewidencji pracowników. Każdy pracownik posiada profil zawierający dane identyfikacyjne, przypisanie do zakładu, rolę systemową, status aktywności oraz opcjonalny identyfikator karty RFID wykorzystywany przy rejestracji czasu pracy.

Profil pracownika stanowi punkt wyjścia dla pozostałych modułów systemu. Z tego poziomu można powiązać pracownika z umowami, certyfikatami, dokumentami, nieobecnościami, projektami oraz wpisami czasu pracy. Takie podejście ogranicza powielanie danych i ułatwia odnalezienie informacji związanych z daną osobą.

3.2. Obsługa struktury organizacyjnej

System powinien umożliwiać odwzorowanie podstawowej struktury organizacyjnej firmy. W projekcie przyjęto podział na zakłady, do których przypisywani są pracownicy, projekty oraz czytniki RCP. Dodatkowo przewidziano mechanizm dostępu użytkowników do wybranych zakładów.

Celem tego rozwiązania jest umożliwienie pracy w organizacji posiadającej więcej niż jedną jednostkę organizacyjną. Dzięki temu użytkownik może mieć dostęp tylko do danych tych zakładów, za które odpowiada.

3.3. Rejestracja i weryfikacja czasu pracy

Istotnym celem projektu jest obsługa czasu pracy. System umożliwia przyjmowanie surowych zdarzeń z czytników RCP, takich jak wejście pracownika do pracy oraz wyjście z pracy. Zdarzenia te są zapisywane w bazie jako rekordy typu `TimeEvent`.

Na podstawie pary zdarzeń IN oraz OUT system tworzy wpis czasu pracy. Wpis ten zawiera godzinę rozpoczęcia, godzinę zakończenia, wyliczoną liczbę godzin, pracownika oraz projekt, z którym powiązany jest czytnik. Dodatkowo wpis czasu pracy posiada status, dzięki czemu można odróżnić dane oczekujące na weryfikację od danych zatwierdzonych.

3.4. Obsługa dokumentacji pracowniczej

Kolejnym celem projektu jest uporządkowanie dokumentacji związanej z pracownikami. System przewiduje obsługę dokumentów, umów, certyfikatów, badań, szkoleń oraz nieobecności.

Dokumenty są przechowywane jako pliki w magazynie obiektowym, natomiast baza danych przechowuje ich metadane oraz powiązania z pracownikiem, umową lub certyfikatem. Takie rozwiązanie oddziela przechowywanie plików od danych relacyjnych, a jednocześnie pozwala zachować logiczne powiązania pomiędzy dokumentami a rekordami systemu.

3.5. Kontrola ważności certyfikatów i badań

System ma wspierać kontrolę terminów ważności certyfikatów, szkoleń, badań lekarskich oraz innych uprawnień pracowników. W tym celu w projekcie przewidziano słownik

certyfikatów oraz tabelę konkretnych certyfikatów przypisanych do pracowników.

Celem tego modułu jest umożliwienie szybkiego sprawdzenia, które dokumenty są aktualne, które zbliżają się do końca ważności oraz które są już przeterminowane. Funkcja ta jest szczególnie istotna w przypadku dokumentów wymaganych do wykonywania określonych zadań lub pracy na konkretnych stanowiskach.

3.6. Rejestracja nieobecności

Projekt obejmuje również obsługę nieobecności pracowników. System pozwala zapisać typ nieobecności, datę rozpoczęcia, datę zakończenia, status zatwierdzenia oraz opcjonalny dokument powiązany z nieobecnością.

Uwzględniono kilka podstawowych typów nieobecności, takich jak urlop, zwolnienie lekarskie, nieobecność nieusprawiedliwiona oraz nieobecność specjalna. Dane te uzupełniają profil pracownika i mogą być wykorzystane podczas analizy obecności oraz planowania pracy.

3.7. Przygotowanie danych do rozliczeń

Celem systemu nie jest zastąpienie pełnego systemu księgowego, lecz przygotowanie uporządkowanych danych, które mogą zostać wykorzystane przez osoby odpowiedzialne za rozliczenia. Podstawą tych danych są zatwierdzone wpisy czasu pracy oraz informacje o aktualnych umowach pracowników.

System umożliwia utworzenie eksportu rozliczeniowego, który grupuje zatwierdzone wpisy czasu pracy dla danego okresu. Dzięki temu można ograniczyć ryzyko wielokrotnego rozliczenia tych samych godzin oraz zachować informację o tym, kto wygenerował daną paczkę rozliczeniową.

3.8. Kontrola dostępu i role użytkowników

W projekcie zastosowano podział użytkowników według ról. System przewiduje między innymi role administratora, pracownika HR, księgowości, brygadzysty, pracownika oraz użytkownika biurowego.

Celem tego mechanizmu jest ograniczenie dostępu do funkcji zgodnie z zakresem odpowiedzialności danej osoby. Administrator posiada najszerszy zakres uprawnień, HR odpowiada za dane kadrowe, księgowość za dane rozliczeniowe, a brygadzysta za

informacje związane z pracownikami i projektami przypisanymi do jego obszaru działania.

3.9. Cel techniczny projektu

Oprócz realizacji wymagań funkcjonalnych projekt ma również cel techniczny. System został przygotowany jako aplikacja działająca w środowisku kontenerowym. W skład środowiska wchodzi: frontend, backend, baza PostgreSQL, magazyn plików MinIO oraz Nginx pełniący rolę reverse proxy.

Takie podejście ułatwia uruchomienie aplikacji w powtarzalny sposób. Projekt zawiera również konfigurację środowiska, przykładowy plik zmiennych oraz skrypty seedujące bazę. Dzięki temu możliwe jest uruchomienie czystej instancji systemu albo instancji zawierającej dane demonstracyjne potrzebne do prezentacji.

3.10. Zakładany rezultat

Rezultatem projektu ma być działająca aplikacja webowa, która prezentuje spójny model zarządzania pracownikami, czasem pracy, dokumentacją i rozliczeniami. System powinien umożliwiać przejście przez podstawowy proces: od utworzenia pracownika, przez przypisanie go do zakładu i projektu, rejestrację czasu pracy, obsługę dokumentów i certyfikatów, aż po przygotowanie danych do rozliczeń.

Projekt stanowi podstawę do dalszego rozwoju. W przyszłości może zostać rozszerzony między innymi o bardziej zaawansowane raportowanie, powiadomienia, integrację z zewnętrznym systemem księgowym lub zewnętrznym dostawcą tożsamości.

4. Analiza wymagań

Analiza wymagań stanowi jeden z kluczowych etapów projektowania systemu informatycznego. Jej celem jest określenie, jakie funkcje powinien realizować system **WorkFlow**, jakie ograniczenia techniczne należy uwzględnić oraz w jaki sposób poszczególne moduły aplikacji powinny ze sobą współpracować.

W przypadku projektu **WorkFlow** wymagania wynikają z potrzeby uporządkowania procesów związanych z obsługą pracowników, czasu pracy, dokumentacji kadrowej oraz danych wykorzystywanych przy rozliczeniach. System nie jest projektowany jako pełny system ERP, lecz jako aplikacja wspierająca wybrany, jasno określony zakres procesów

kadrowo-organizacyjnych.

4.1. Podejście do analizy

Wymagania zostały opracowane z podziałem na trzy główne grupy: wymagania nie-funkcyjne, funkcjonalne oraz integracyjne. Taki podział pozwala oddzielić opis konkretnych funkcji systemu od cech jakościowych i technicznych, które wpływają na sposób działania aplikacji.

W analizie przyjęto, że system powinien być możliwy do uruchomienia w środowisku kontenerowym oraz składać się z kilku niezależnych usług. Pozwala to na czytelny podział odpowiedzialności pomiędzy warstwę interfejsu użytkownika, logikę biznesową, bazę danych, magazyn plików oraz reverse proxy.

Z punktu widzenia użytkownika końcowego najważniejsze jest to, aby system umożliwiał sprawną obsługę podstawowych operacji: zarządzanie pracownikami, przypisywanie ich do zakładów i projektów, rejestrowanie czasu pracy, kontrolowanie certyfikatów, przechowywanie dokumentów, obsługę nieobecności oraz przygotowanie danych rozliczeniowych.

4.2. Zakres wymagań

Zakres wymagań został dostosowany do aktualnego stanu projektu oraz funkcji przewidzianych do prezentacji. Obejmuje on następujące obszary:

- obsługę kont użytkowników i ról systemowych,
- zarządzanie pracownikami oraz ich przypisaniem do zakładów,
- zarządzanie projektami i przypisaniami pracowników,
- rejestrowanie zdarzeń czasu pracy z czytników RCP,
- tworzenie i weryfikację wpisów czasu pracy,
- obsługę umów pracowników,
- obsługę certyfikatów, badań i szkoleń,
- rejestrowanie nieobecności,
- przechowywanie dokumentów pracowniczych,

- przygotowanie danych do rozliczeń,
- uruchomienie systemu w środowisku Docker Compose.

Z zakresu projektu wyłączono moduły, które nie są częścią aktualnej wersji aplikacji. Dotyczy to w szczególności funkcji związanych z obsługą wypożyczonego sprzętu. Pozwala to zachować spójność pomiędzy dokumentacją, modelem danych prezentowanym w diagramach oraz rzeczywistym zakresem systemu.

4.3. Założenia techniczne

Podczas definiowania wymagań przyjęto kilka założeń technicznych, które mają wpływ na dalszy projekt systemu:

- aplikacja działa jako system webowy dostępny z poziomu przeglądarki internetowej,
- frontend i backend są oddzielnymi komponentami,
- backend udostępnia API wykorzystywane przez frontend oraz przez techniczny endpoint czytników RCP,
- dane relacyjne są przechowywane w bazie PostgreSQL,
- pliki dokumentów są przechowywane poza bazą danych, w magazynie obiektowym MinIO,
- komunikacja użytkownika z systemem odbywa się przez Nginx pełniący rolę reverse proxy,
- konfiguracja środowiska jest oparta o plik `.env`,
- projekt posiada dane demonstracyjne umożliwiające szybkie zaprezentowanie działania systemu.

Przyjęta architektura ułatwia rozdzielenie odpowiedzialności pomiędzy poszczególne części systemu. Warstwa frontendowa odpowiada za prezentację danych, warstwa backendowa za logikę biznesową i autoryzację, baza danych za trwałe przechowywanie informacji, a magazyn plików za obsługę dokumentów.

4.4. Podział wymagań

Dalsza część rozdziału została podzielona na trzy podrozdziały:

1. **Wymagania niefunkcyjne** – opisujące cechy jakościowe systemu, takie jak bezpieczeństwo, kontrola dostępu, przenośność środowiska oraz spójność danych.
2. **Wymagania funkcjonalne** – opisujące konkretne operacje wykonywane przez użytkowników oraz główne moduły aplikacji.
3. **Wymagania integracyjne** – opisujące sposób współpracy systemu z bazą danych, magazynem plików, czytnikami RCP oraz usługami infrastrukturalnymi.

Taki układ pozwala uporządkować opis systemu i ułatwia powiązanie wymagań z późniejszym modelem danych, przypadkami użycia oraz scenariuszami działania.

4.5. Wymagania niefunkcyjne

Wymagania niefunkcyjne określają cechy jakościowe systemu **WorkFlow**, które wpływają na sposób jego działania, bezpieczeństwo, utrzymanie oraz możliwość uruchomienia w przygotowanym środowisku. Nie opisują one pojedynczych funkcji użytkownika, lecz warunki, jakie powinny zostać spełnione przez całą aplikację.

W przypadku systemu **WorkFlow** szczególne znaczenie mają: bezpieczeństwo danych pracowników, kontrola dostępu, spójność danych, możliwość uruchomienia systemu w środowisku kontenerowym oraz czytelność interfejsu użytkownika.

4.5.1. Bezpieczeństwo danych

System przetwarza dane pracowników, informacje o umowach, certyfikatach, nieobecnościach, dokumentach oraz czasie pracy. Z tego względu dostęp do aplikacji powinien być ograniczony do zalogowanych użytkowników.

Podstawowe wymagania bezpieczeństwa obejmują:

- uwierzytelnianie użytkowników za pomocą adresu e-mail i hasła,
- przechowywanie haseł w postaci zahashowanej,
- stosowanie tokenów JWT do autoryzacji żądań,
- ograniczenie dostępu do funkcji systemu na podstawie roli użytkownika,

- ograniczenie dostępu do danych zakładów zgodnie z przypisanymi uprawnieniami,
- zabezpieczenie technicznego endpointu czytników RCP osobnym tokenem urządzenia.

W środowisku produkcyjnym system powinien być uruchamiany za pośrednictwem połączenia szyfrowanego HTTPS. W środowisku lokalnym dopuszczalne jest uruchomienie po protokole HTTP, co zostało uwzględnione w konfiguracji plików środowiskowych.

4.5.2. Kontrola dostępu i role użytkowników

System powinien realizować kontrolę dostępu opartą na rolach użytkowników. Każda rola odpowiada innemu zakresowi odpowiedzialności w systemie.

W projekcie przewidziano następujące role:

- **Administrator** – posiada najszerszy zakres dostępu do systemu,
- **HR** – odpowiada za obsługę danych kadrowych, pracowników, umów, certyfikatów, dokumentów i nieobecności,
- **Księgowość** – korzysta z danych potrzebnych do rozliczeń,
- **Brygadzysta** – ma dostęp do danych związanych z pracownikami, projektami i zakładem, za który odpowiada,
- **Pracownik** – reprezentuje podstawowego użytkownika systemu,
- **Użytkownik biurowy** – rola pomocnicza przewidziana dla prac administracyjnych.

Takie rozdzielenie dostępu pozwala ograniczyć widoczność danych wrażliwych oraz zmniejsza ryzyko przypadkowej modyfikacji informacji przez użytkowników, którzy nie powinni mieć dostępu do danego obszaru systemu.

4.5.3. Spójność i integralność danych

Dane systemu powinny być przechowywane w relacyjnej bazie danych w sposób zapewniający spójność pomiędzy encjami. Pracownik jest powiązany z zakładem, projektami, umowami, certyfikatami, dokumentami, nieobecnościami oraz wpisami czasu pracy. Z tego względu istotne jest zachowanie poprawnych relacji pomiędzy tabelami.

Wymagania w tym obszarze obejmują:

- stosowanie jednoznacznych identyfikatorów rekordów,
- wykorzystywanie kluczy obcych do opisu relacji pomiędzy tabelami,
- ograniczenie duplikowania danych,
- przechowywanie historii wybranych informacji, na przykład umów i przypisań do projektów,
- oddzielenie surowych zdarzeń czasu pracy od przetworzonych wpisów czasu pracy.

Szczególnie ważne jest rozróżnienie pomiędzy tabelą `TimeEvent` a tabelą `TimeEntry`. Pierwsza z nich przechowuje surowe zdarzenia z czytników RCP, natomiast druga zawiera przetworzone wpisy czasu pracy utworzone na podstawie pary zdarzeń wejścia i wyjścia.

4.5.4. Przenośność środowiska

System powinien być możliwy do uruchomienia w powtarzalny sposób na różnych środowiskach. W tym celu wykorzystano konteneryzację oraz plik Docker Compose, który definiuje zestaw usług potrzebnych do działania aplikacji.

Środowisko uruchomieniowe powinno obejmować:

- kontener frontendu,
- kontener backendu,
- kontener bazy danych PostgreSQL,
- kontener magazynu plików MinIO,
- kontener Nginx pełniący funkcję reverse proxy.

Takie podejście upraszcza uruchomienie projektu oraz ogranicza liczbę ręcznych kroków konfiguracyjnych. Użytkownik nie musi oddzielnie instalować i konfigurować każdej usługi, ponieważ ich podstawowa konfiguracja znajduje się w pliku `docker-compose.prod.yml` oraz pliku środowiskowym `.env`.

4.5.5. Konfigurowalność

System powinien umożliwiać zmianę podstawowych parametrów środowiska bez modyfikowania kodu źródłowego. W tym celu wykorzystano zmienne środowiskowe zapisane w pliku `.env`.

Do najważniejszych parametrów konfiguracyjnych należą:

- dane dostępowe do bazy PostgreSQL,
- sekret JWT,
- token techniczny urządzeń RCP,
- dane dostępowe do MinIO,
- adres API wykorzystywany przez frontend,
- porty usług,
- ustawienia związane z ciasteczkami autoryzacyjnymi,
- parametry seedów inicjalizujących dane.

Do repozytorium nie powinien trafiać rzeczywisty plik `.env`. Zamiast niego przechowywany jest plik `.env.example`, który opisuje wymagane zmienne bez ujawniania haseł i tokenów.

4.5.6. Użyteczność interfejsu

Interfejs użytkownika powinien być czytelny i dostosowany do typowych zadań administracyjnych. Użytkownik powinien mieć możliwość szybkiego przejścia do najważniejszych modułów, takich jak pracownicy, zakłady, projekty, czas pracy, certyfikaty czy nieobecności.

Wymagania dotyczące użyteczności obejmują:

- przejrzysty układ nawigacji,
- możliwość wyszukiwania i filtrowania danych,
- czytelne formularze dodawania i edycji rekordów,
- spójny wygląd widoków w trybie jasnym i ciemnym,

- wyraźne oznaczanie statusów, na przykład aktywności pracownika, ważności certyfikatu lub statusu wpisu czasu pracy.

System powinien działać w aktualnych wersjach popularnych przeglądarek internetowych. Ponieważ aplikacja ma charakter administracyjny, podstawowym środowiskiem pracy jest komputer z przeglądarką internetową.

4.5.7. Niezawodność i odtwarzalność danych demonstracyjnych

Aplikacja powinna umożliwiać ponowne uruchomienie usług bez konieczności ręcznego odtwarzania całej konfiguracji. Dane bazy i magazynu plików powinny być przechowywane w wolumenach Docker, dzięki czemu nie są usuwane przy zwykłym restarcie kontenerów.

Na potrzeby prezentacji projektu przygotowano również dane demonstracyjne. Seed demonstracyjny pozwala odtworzyć przykładowy stan systemu zawierający użytkowników, zakłady, pracowników, projekty, umowy, certyfikaty, nieobecności, dokumenty oraz wpisy czasu pracy. Dzięki temu możliwe jest szybkie przygotowanie środowiska pokazowego bez ręcznego wprowadzania wszystkich danych.

4.6. Wymagania funkcjonalne

Wymagania funkcjonalne określają operacje, które system WorkFlow powinien udostępniać użytkownikom oraz procesy, które powinny być obsługiwane przez aplikację. Wymagania zostały pogrupowane według głównych modułów systemu, aby zachować zgodność pomiędzy opisem funkcjonalnym, przypadkami użycia, modelem danych oraz implementacją.

Zakres funkcjonalny systemu obejmuje obsługę pracowników, zakładów, projektów, czasu pracy, umów, certyfikatów, nieobecności, dokumentów oraz danych rozliczeniowych. Dodatkowo system powinien zapewniać logowanie użytkowników, kontrolę dostępu oraz możliwość uruchomienia danych demonstracyjnych potrzebnych do prezentacji projektu.

4.6.1. Logowanie i obsługa użytkowników

System powinien umożliwiać użytkownikowi zalogowanie się do aplikacji przy użyciu adresu e-mail oraz hasła. Po poprawnym logowaniu użytkownik otrzymuje dostęp do panelu systemu zgodnie ze swoją rolą oraz zakresem dostępu do zakładów.

Wymagania w tym obszarze obejmują:

- logowanie użytkownika za pomocą adresu e-mail i hasła,
- wylogowanie użytkownika z systemu,
- przechowywanie informacji o zalogowanym użytkowniku w ramach sesji,
- rozpoznawanie roli użytkownika,
- ograniczanie dostępu do wybranych widoków i operacji na podstawie roli,
- obsługę dostępu użytkownika do jednego lub wielu zakładów.

Mechanizm logowania stanowi punkt wejścia do systemu i jest powiązany z modelem pracownika, ponieważ użytkownik aplikacji jest jednocześnie rekordem w tabeli pracowników.

4.6.2. Zarządzanie zakładami

System powinien umożliwiać prowadzenie ewidencji zakładów, które reprezentują jednostki organizacyjne firmy. Zakład jest nadrzędnym elementem struktury organizacyjnej, do którego mogą być przypisani pracownicy, projekty oraz czytniki RCP.

Wymagania funkcjonalne dla modułu zakładów obejmują:

- dodawanie nowych zakładów,
- edycję danych zakładu,
- wyświetlanie listy zakładów,
- oznaczanie zakładu jako aktywny lub nieaktywny,
- przypisywanie pracowników do zakładu,
- przypisywanie projektów do zakładu,
- obsługę dostępu użytkowników do wybranych zakładów.

Dzięki temu system może obsługiwać organizację posiadającą więcej niż jedną lokalizację lub jednostkę administracyjną.

4.6.3. Zarządzanie pracownikami

Jednym z głównych modułów systemu jest moduł pracowników. Powinien on umożliwiać prowadzenie centralnej ewidencji osób zatrudnionych lub obsługiwanych w systemie.

System powinien umożliwiać:

- dodawanie pracowników,
- edycję danych pracownika,
- przeglądanie listy pracowników,
- wyszukiwanie i filtrowanie pracowników,
- przejście do szczegółowego profilu pracownika,
- oznaczenie pracownika jako aktywnego lub nieaktywnego,
- przypisanie pracownika do zakładu,
- przypisanie pracownikowi roli systemowej,
- zapis identyfikatora karty RFID wykorzystywanej przy rejestracji czasu pracy.

Profil pracownika powinien gromadzić informacje powiązane z daną osobą. Obejmuje to dane podstawowe, umowy, certyfikaty, dokumenty, nieobecności, przypisania do projektów oraz wpisy czasu pracy. Taki układ pozwala traktować profil pracownika jako centralne miejsce dostępu do danych kadrowych.

4.6.4. Zarządzanie projektami

System powinien umożliwiać prowadzenie ewidencji projektów. Projekt reprezentuje miejsce, kontrakt lub obszar pracy, do którego mogą być przypisywani pracownicy i z którym mogą być powiązane wpisy czasu pracy.

Wymagania w zakresie projektów obejmują:

- tworzenie projektów,
- przypisywanie projektu do zakładu,
- określanie nazwy, kodu wewnętrznego oraz adresu projektu,

- oznaczanie statusu projektu,
- określanie daty rozpoczęcia i zakończenia,
- przypisywanie pracowników do projektu,
- przechowywanie historii przypisań pracowników do projektów.

Status projektu powinien umożliwiać rozróżnienie projektów planowanych, aktywnych, zakończonych oraz wstrzymanych. Informacja ta jest istotna przy filtrowaniu projektów oraz przy analizie czasu pracy.

4.6.5. Przypisania pracowników do projektów

System powinien umożliwiać powiązanie pracownika z projektem. Przypisanie powinno zawierać datę rozpoczęcia, opcjonalną datę zakończenia oraz dodatkowe uwagi.

Wymagania w tym zakresie obejmują:

- przypisanie pracownika do projektu,
- zapis daty przypisania,
- możliwość zakończenia przypisania,
- przechowywanie historii przypisań,
- prezentację przypisań w profilu pracownika lub widoku projektu.

Mechanizm przypisań pozwala odwzorować fakt, że jeden pracownik może pracować przy różnych projektach w różnych okresach.

4.6.6. Rejestracja zdarzeń czasu pracy

System powinien przyjmować zdarzenia czasu pracy pochodzące z czytników RCP. Czytnik jest powiązany z zakładem oraz projektem, dzięki czemu zdarzenie odbicia karty może zostać przypisane do właściwego miejsca pracy.

Zdarzenie czasu pracy powinno zawierać:

- numer seryjny czytnika,
- identyfikator karty RFID pracownika,

- typ zdarzenia: IN lub OUT,
- dokładny czas zdarzenia,
- opcjonalny zewnętrzny identyfikator zdarzenia.

Po odebraniu zdarzenia system powinien sprawdzić, czy istnieje aktywny czytnik oraz pracownik o podanym identyfikatorze karty RFID. Surowe zdarzenie powinno zostać zapisane w bazie danych jako rekord `TimeEvent`.

4.6.7. Tworzenie wpisów czasu pracy

Na podstawie zdarzeń IN oraz OUT system powinien tworzyć przetworzone wpisy czasu pracy. Wpis czasu pracy powinien zawierać pracownika, projekt, godzinę rozpoczęcia, godzinę zakończenia, wyliczoną liczbę godzin oraz status.

Wymagania w tym obszarze obejmują:

- wyszukanie ostatniego zdarzenia IN dla pracownika,
- utworzenie wpisu czasu pracy po odebraniu zdarzenia OUT,
- powiązanie wpisu z projektem wynikającym z czytnika RCP,
- wyliczenie liczby przepracowanych godzin,
- oznaczenie wpisu statusem początkowym,
- możliwość późniejszej weryfikacji lub zatwierdzenia wpisu.

Rozdzielenie tabel `TimeEvent` i `TimeEntry` pozwala zachować zarówno surowe dane z urządzenia, jak i dane przetworzone wykorzystywane później w raportowaniu oraz rozliczeniach.

4.6.8. Obsługa umów

System powinien umożliwiać przechowywanie informacji o umowach pracowników. Umowa zawiera typ, kwotę wynagrodzenia lub stawkę, datę rozpoczęcia, opcjonalną datę zakończenia oraz informację, czy jest aktualna.

Wymagania funkcjonalne obejmują:

- dodanie umowy pracownika,

- edycję danych umowy,
- przechowywanie typu umowy,
- zapis kwoty wynagrodzenia,
- określenie daty rozpoczęcia i zakończenia,
- oznaczenie umowy jako aktualnej,
- przechowywanie historii wcześniejszych umów.

Historia umów jest istotna, ponieważ warunki zatrudnienia mogą zmieniać się w czasie. Zachowanie poprzednich rekordów pozwala analizować dane pracownika w kontekście okresu obowiązywania danej umowy.

4.6.9. Obsługa certyfikatów, badań i szkoleń

System powinien umożliwiać obsługę certyfikatów, badań lekarskich, szkoleń BHP, uprawnień UDT oraz innych dokumentów potwierdzających kwalifikacje pracownika.

Moduł powinien obejmować:

- zarządzanie słownikiem certyfikatów,
- określenie typu certyfikatu,
- zapis domyślnego okresu ważności,
- przypisywanie certyfikatu do pracownika,
- zapis numeru certyfikatu lub zaświadczenia,
- określenie daty wydania,
- określenie daty wygaśnięcia,
- prezentację certyfikatów wygasających w wybranym okresie.

Funkcja monitorowania dat ważności pozwala szybciej wykryć certyfikaty, które wymagają odnowienia. Jest to szczególnie ważne w przypadku dokumentów wymaganych do wykonywania określonych obowiązków.

4.6.10. Obsługa nieobecności

System powinien umożliwiać rejestrowanie nieobecności pracowników. Nieobecność powinna zawierać typ, datę rozpoczęcia, datę zakończenia, status zatwierdzenia oraz opcjonalny dokument.

Wymagania funkcjonalne dla nieobecności obejmują:

- dodanie nieobecności pracownika,
- określenie typu nieobecności,
- wskazanie zakresu dat,
- oznaczenie nieobecności jako zatwierdzonej lub oczekującej,
- powiązanie nieobecności z dokumentem,
- wyświetlanie nieobecności w profilu pracownika.

W systemie przewidziano typy nieobecności takie jak urlop, zwolnienie lekarskie, nieobecność nieusprawiedliwiona oraz nieobecność specjalna.

4.6.11. Obsługa dokumentów

System powinien umożliwiać przechowywanie informacji o dokumentach pracowników. Plik dokumentu powinien być przechowywany w magazynie obiektowym, natomiast baza danych powinna przechowywać metadane pliku oraz powiązania z innymi rekordami.

Wymagania w tym obszarze obejmują:

- zapis nazwy pliku,
- zapis lokalizacji pliku w magazynie obiektowym,
- powiązanie dokumentu z pracownikiem,
- opcjonalne powiązanie dokumentu z umową,
- opcjonalne powiązanie dokumentu z certyfikatem,
- opcjonalne powiązanie dokumentu z nieobecnością.

Takie rozwiązanie pozwala utrzymać logiczne powiązanie dokumentów z danymi kadrowymi bez przechowywania plików bezpośrednio w tabelach relacyjnych.

4.6.12. Rozliczenia i eksport danych

System powinien wspierać przygotowanie danych potrzebnych do rozliczeń. Podstawą rozliczeń są zatwierdzone wpisy czasu pracy oraz informacje o pracownikach i ich umowach.

Wymagania funkcjonalne obejmują:

- zebranie wpisów czasu pracy z wybranego okresu,
- wykorzystanie wyłącznie danych zatwierdzonych,
- utworzenie eksportu rozliczeniowego,
- zapis miesiąca i roku rozliczeniowego,
- zapis informacji o użytkowniku generującym eksport,
- powiązanie wpisów czasu pracy z eksportem,
- ograniczenie ryzyka ponownego rozliczenia tych samych godzin.

Moduł rozliczeniowy nie zastępuje pełnego systemu księgowego. Jego zadaniem jest przygotowanie uporządkowanych danych, które mogą zostać wykorzystane przez osobę odpowiedzialną za rozliczenia.

4.6.13. Audit log

System powinien umożliwiać rejestrowanie wybranych operacji wykonywanych na danych. Rejestr audytowy powinien zawierać informację o użytkowniku wykonującym operację, nazwie akcji, encji, identyfikatorze rekordu oraz opcjonalnie poprzednich i nowych wartościach.

Wymagania w tym zakresie obejmują:

- zapis użytkownika wykonującego operację,
- zapis rodzaju wykonanej akcji,
- zapis nazwy encji, której dotyczy operacja,
- zapis identyfikatora modyfikowanego rekordu,
- możliwość przechowywania wartości przed zmianą i po zmianie.

Mechanizm audytu zwiększa przejrzystość pracy systemu i ułatwia analizę zmian w danych wrażliwych, takich jak umowy lub dane pracowników.

4.6.14. Dane inicjalizacyjne i demonstracyjne

System powinien posiadać mechanizmy ułatwiające uruchomienie nowego środowiska. W projekcie przewidziano dwa rodzaje danych inicjalizacyjnych.

Pierwszy z nich to seed minimalny, który tworzy podstawowy zakład oraz konta startowe dla głównych ról użytkowników. Drugi to seed demonstracyjny, który generuje pełniejszy zestaw danych potrzebnych do prezentacji działania systemu.

Dane demonstracyjne powinny obejmować:

- konta użytkowników,
- zakłady,
- pracowników,
- projekty,
- czytniki RCP,
- umowy,
- certyfikaty,
- dokumenty,
- nieobecności,
- zdarzenia i wpisy czasu pracy,
- przykładowy eksport rozliczeniowy.

Dzięki temu możliwe jest szybkie przygotowanie środowiska do testów lub prezentacji bez ręcznego wprowadzania danych.

4.7. Wymagania integracyjne

Wymagania integracyjne opisują sposób współpracy systemu WorkFlow z usługami technicznymi, komponentami infrastruktury oraz zewnętrznymi źródłami zdarzeń. W

aktualnym zakresie projektu integracje dotyczą przede wszystkim bazy danych, magazynu plików, czytników RCP, komunikacji pomiędzy frontendem i backendem oraz środowiska uruchomieniowego opartego o kontenery.

Celem wymagań integracyjnych jest określenie, jakie elementy muszą ze sobą współpracować, aby system działał jako kompletna aplikacja webowa. Wymagania te nie opisują pojedynczych funkcji użytkownika, lecz zależności pomiędzy usługami i warstwami systemu.

4.7.1. Integracja z bazą danych

System powinien integrować się z relacyjną bazą danych PostgreSQL. Baza danych stanowi główne miejsce przechowywania informacji biznesowych, takich jak pracownicy, zakłady, projekty, umowy, certyfikaty, nieobecności, dokumenty, zdarzenia czasu pracy, wpisy czasu pracy oraz eksporty rozliczeniowe.

Komunikacja backendu z bazą danych odbywa się z wykorzystaniem narzędzia Prisma ORM. Schemat bazy został zdefiniowany w pliku `schema.prisma`, co umożliwia utrzymanie spójności pomiędzy modelem danych aplikacji a strukturą fizycznej bazy danych.

Wymagania w tym obszarze obejmują:

- możliwość połączenia backendu z bazą PostgreSQL za pomocą zmiennej środowiskowej,
- odwzorowanie encji biznesowych w modelach Prisma,
- obsługę relacji pomiędzy tabelami,
- możliwość synchronizacji struktury bazy danych ze schematem Prisma,
- możliwość inicjalizacji bazy danymi startowymi.

4.7.2. Integracja z magazynem plików

System powinien współpracować z magazynem obiektowym MinIO. Pliki dokumentów nie są przechowywane bezpośrednio w tabelach bazy danych. W bazie zapisywane są jedynie metadane dokumentu, takie jak nazwa pliku, właściciel dokumentu oraz ścieżka do pliku w magazynie.

Takie rozwiązanie pozwala oddzielić dane relacyjne od plików binarnych. Jest to szczególnie istotne w przypadku dokumentów kadrowych, skanów umów, zaświadczeń, certyfikatów lub dokumentów powiązanych z nieobecnościami.

Wymagania w zakresie integracji z magazynem plików obejmują:

- konfigurację endpointu MinIO za pomocą zmiennych środowiskowych,
- obsługę danych dostępowych do magazynu plików,
- wskazanie nazwy bucketu przechowującego dokumenty,
- zapisywanie w bazie danych ścieżki lub adresu pliku,
- możliwość powiązania dokumentu z pracownikiem, umową, certyfikatem lub nieobecnością.

4.7.3. Integracja z czytnikami RCP

System powinien udostępniać techniczny endpoint przeznaczony do odbierania zdarzeń z czytników RCP. Czytnik jest powiązany z zakładem oraz projektem, dzięki czemu zdarzenia wejścia i wyjścia mogą zostać przypisane do konkretnego pracownika oraz miejsca pracy.

Zdarzenie przekazywane do systemu powinno zawierać:

- numer seryjny czytnika,
- identyfikator karty RFID,
- typ zdarzenia: IN albo OUT,
- czas zdarzenia,
- opcjonalny zewnętrzny identyfikator zdarzenia.

Endpoint urządzeń powinien być zabezpieczony tokenem technicznym, niezależnym od tokenów użytkowników logujących się do aplikacji. Pozwala to oddzielić komunikację maszynową od standardowej komunikacji użytkownika z systemem.

Po odebraniu zdarzenia backend powinien:

1. zweryfikować token urządzenia,
2. odnaleźć aktywny czytnik na podstawie numeru seryjnego,

3. odnaleźć aktywnego pracownika na podstawie identyfikatora karty RFID,
4. zapisać surowe zdarzenie w tabeli `TimeEvent`,
5. w przypadku zdarzenia `OUT` spróbować utworzyć wpis czasu pracy na podstawie wcześniejszego zdarzenia `IN`.

4.7.4. Komunikacja pomiędzy frontendem i backendem

Frontend powinien komunikować się z backendem za pomocą API HTTP. Backend udostępnia punkty końcowe odpowiedzialne za logowanie, pobieranie danych, tworzenie rekordów, aktualizację informacji oraz obsługę operacji biznesowych.

Frontend odpowiada za:

- prezentację danych użytkownikowi,
- obsługę formularzy,
- wyświetlanie list i szczegółów rekordów,
- kierowanie użytkownika do odpowiednich widoków,
- przekazywanie danych do backendu.

Backend odpowiada za:

- autoryzację żądań,
- walidację danych wejściowych,
- realizację logiki biznesowej,
- komunikację z bazą danych,
- komunikację z magazynem plików,
- obsługę technicznych endpointów, takich jak odbieranie zdarzeń RCP.

Taki podział umożliwia oddzielenie warstwy prezentacji od logiki biznesowej i ułatwia dalsze utrzymanie aplikacji.

4.7.5. Integracja przez Nginx

W środowisku Docker Compose system wykorzystuje Nginx jako reverse proxy. Oznacza to, że użytkownik korzysta z jednego adresu aplikacji, a Nginx przekierowuje ruch do odpowiedniej usługi.

Wymagania dotyczące Nginx obejmują:

- przekierowanie ruchu aplikacji do frontendu,
- przekierowanie zapytań API do backendu,
- uproszczenie adresów wykorzystywanych przez użytkownika,
- oddzielenie wewnętrznych nazw usług od adresu widocznego z poziomu przeglądarki.

Przykładowy podział ruchu wygląda następująco:

- ścieżka / kierowana jest do frontendu,
- ścieżka /api kierowana jest do backendu.

Dzięki temu użytkownik nie musi znać portów poszczególnych usług, a aplikacja może być dostępna pod jednym adresem.

4.7.6. Środowisko Docker Compose

System powinien być możliwy do uruchomienia za pomocą Docker Compose. Plik konfiguracyjny opisuje usługi niezbędne do działania aplikacji oraz zależności pomiędzy nimi.

Środowisko powinno obejmować:

- usługę frontendu,
- usługę backendu,
- usługę PostgreSQL,
- usługę MinIO,
- usługę Nginx.

Konfiguracja powinna korzystać ze zmiennych środowiskowych przechowywanych w pliku `.env`. W repozytorium powinien znajdować się wyłącznie plik przykładowy `.env.example`, który opisuje wymagane zmienne bez ujawniania rzeczywistych haseł, tokenów i sekretów.

4.7.7. Inicjalizacja danych

System powinien umożliwiać przygotowanie bazy danych do pracy po pierwszym uruchomieniu. W projekcie przewidziano dwa mechanizmy inicjalizacji danych:

- **seed minimalny** – tworzący podstawowy zakład oraz konta startowe dla głównych ról użytkowników,
- **seed demonstracyjny** – tworzący komplet danych potrzebnych do prezentacji działania systemu.

Seed minimalny jest przeznaczony do czystego uruchomienia aplikacji. Seed demonstracyjny służy natomiast do przygotowania środowiska pokazowego i zawiera między innymi pracowników, projekty, czytniki RCP, umowy, certyfikaty, nieobecności, dokumenty, wpisy czasu pracy oraz przykładowy eksport rozliczeniowy.

Ze względu na to, że seed demonstracyjny może czyścić istniejące dane, jego uruchomienie powinno wymagać dodatkowej zmiennej środowiskowej potwierdzającej intencję wykonania operacji.

4.7.8. Integracje możliwe do rozbudowy

Aktualna wersja systemu skupia się na podstawowych integracjach wymaganych do działania aplikacji. Przyjęta architektura pozwala jednak na późniejsze rozszerzenie systemu o kolejne mechanizmy.

Do możliwych kierunków rozwoju należą:

- integracja z zewnętrznym systemem księgowym,
- wysyłka powiadomień e-mail,
- integracja z zewnętrznym dostawcą tożsamości,
- rozbudowa harmonogramów zadań cyklicznych,
- bardziej zaawansowany eksport danych rozliczeniowych.

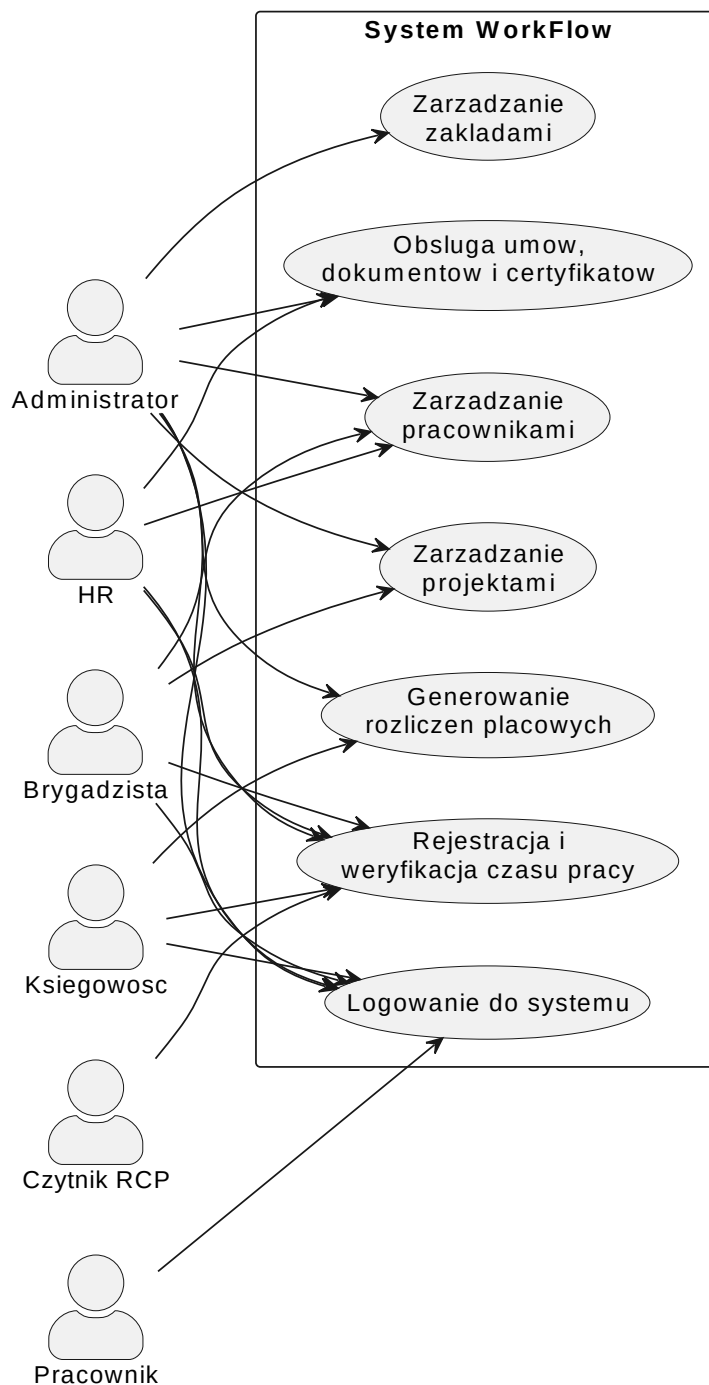
Wymienione elementy nie są wymagane do działania aktualnej wersji projektu. Zostały wskazane jako potencjalne rozszerzenia, które mogą zostać dodane w przyszłości bez zmiany podstawowych założeń architektonicznych systemu.

5. Diagram przypadkow uzycia

W celu poprawy czytelności zamiast jednego rozbudowanego diagramu przygotowano cztery osobne diagramy przypadkow uzycia. Pierwszy przedstawia ogólny obraz systemu, natomiast kolejne trzy koncentrują się na najważniejszych obszarach funkcjonalnych aplikacji.

5.1. Diagram ogólny systemu

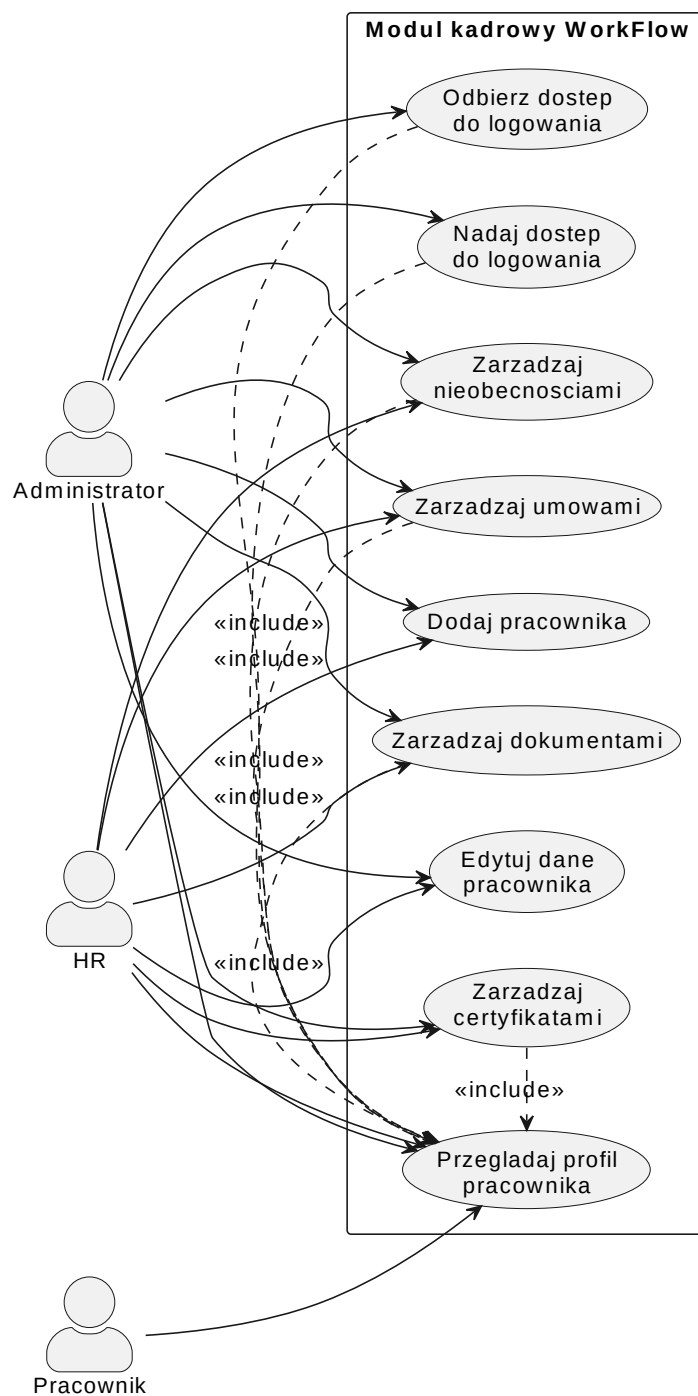
Diagram ogólny przypadków użycia systemu WorkFlow



Rys. 5.1: Ogólny diagram przypadków użycia systemu WorkFlow

5.2. Modul kadrowy

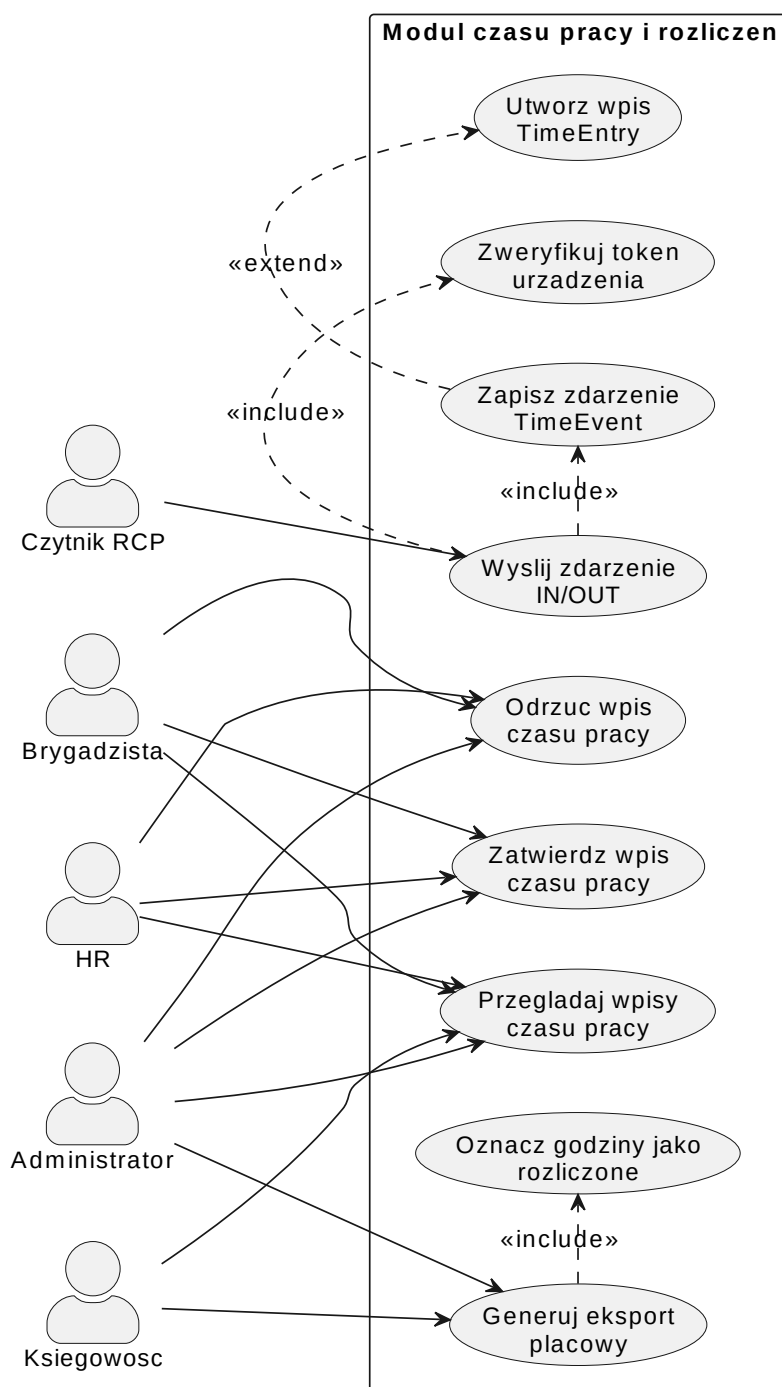
Diagram przypadków użycia - modul kadrowy



Rys. 5.2: Diagram przypadków użycia dla modułu kadrowego

5.3. Czas pracy, RCP i rozliczenia

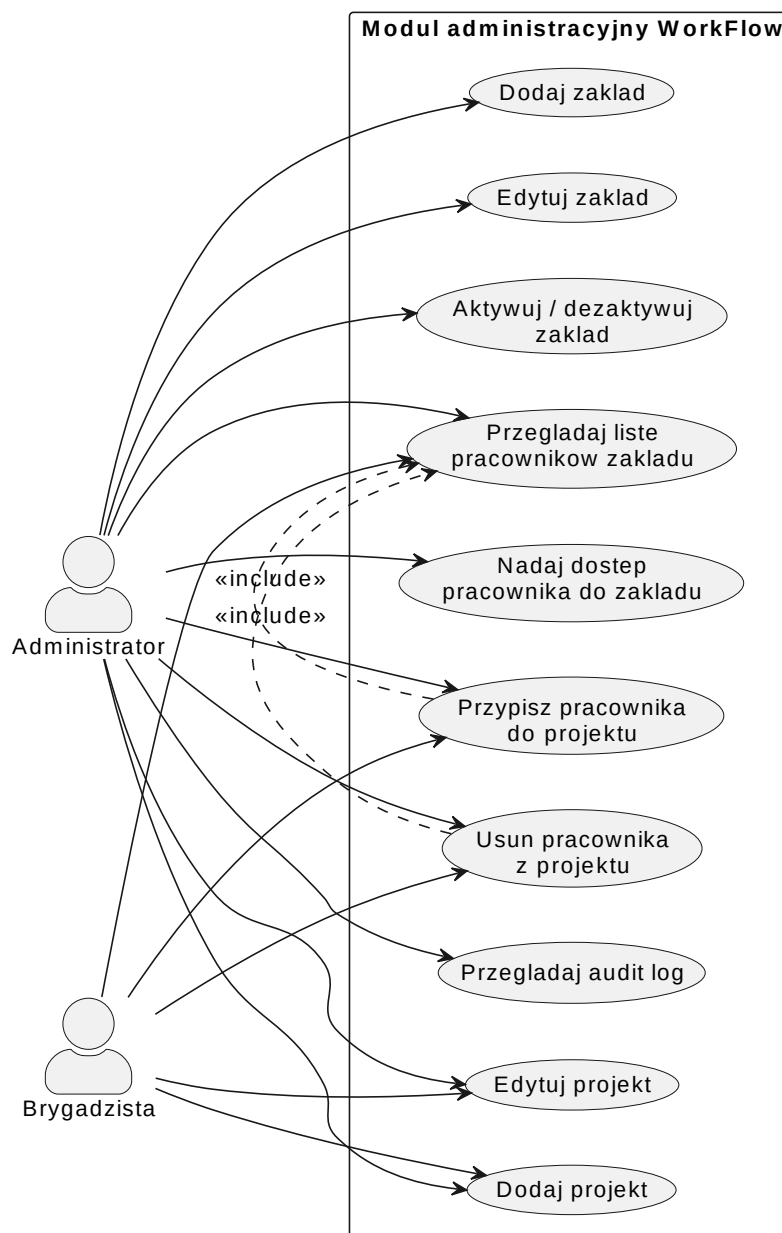
Diagram przypadków użycia - czas pracy, RCP i payroll



Rys. 5.3: Diagram przypadków użycia dla rejestracji czasu pracy i rozliczeń

5.4. Administracja i struktura organizacyjna

Diagram przypadków użycia - administracja i struktura organizacyjna



Rys. 5.4: Diagram przypadków użycia dla administracji i zarządzania strukturą organizacyjną

5.5. Aktorzy systemu

W systemie wyróżniono aktorów odpowiadających rolom użytkowników oraz jeden aktor techniczny reprezentujący czytnik RCP. Każdy aktor korzysta z innego zakresu

funkcji, zgodnie z odpowiedzialnością w procesie obsługi pracowników, czasu pracy i dokumentacji.

- **Administrator** – użytkownik posiadający najszerszy zakres uprawnień. Odpowiada za ogólną konfigurację systemu, dostęp do zakładów, role użytkowników oraz kontrolę poprawności danych.
- **HR** – użytkownik odpowiedzialny za obsługę danych kadrowych. Zarządza pracownikami, umowami, certyfikatami, dokumentami oraz nieobecnościami.
- **Księgowość** – użytkownik korzystający z danych potrzebnych do rozliczeń. Pracuje głównie na zatwierdzonych wpisach czasu pracy i eksportach rozliczeniowych.
- **Brygadzysta** – użytkownik związany z pracą operacyjną. Ma dostęp do informacji dotyczących wybranych zakładów, projektów i pracowników.
- **Pracownik** – podstawowa rola osoby znajdującej się w ewidencji systemu. W obecnym zakresie projektu rola ta pełni przede wszystkim funkcję danych pracowniczych, które są wykorzystywane przez inne moduły.
- **Czytnik RCP** – aktor techniczny reprezentujący urządzenie wysyłające zdarzenia wejścia i wyjścia pracownika. Nie jest użytkownikiem interfejsu webowego, ale komunikuje się z backendem przez techniczny endpoint API.

5.6. Zakres przypadków użycia

Najważniejsze przypadki użycia systemu można podzielić na kilka grup funkcjonalnych:

- **Uwierzytelnianie i dostęp** – logowanie użytkownika, wylogowanie, sprawdzenie roli oraz zakresu dostępu do zakładów.
- **Zarządzanie strukturą organizacyjną** – obsługa zakładów, przypisywanie pracowników do zakładów oraz nadawanie dostępu użytkownikom do wybranych jednostek.
- **Zarządzanie pracownikami** – dodawanie, edycja, wyszukiwanie i przeglądanie pracowników oraz ich profili.
- **Zarządzanie projektami** – tworzenie projektów, oznaczanie ich statusów oraz przypisywanie pracowników do projektów.

- **Rejestracja czasu pracy** – odbieranie zdarzeń z czytników RCP, zapisywanie zdarzeń IN/OUT oraz tworzenie wpisów czasu pracy.
- **Obsługa dokumentacji kadrowej** – obsługa umów, dokumentów, certyfikatów, badań, szkoleń i nieobecności.
- **Rozliczenia** – przygotowanie danych rozliczeniowych oraz tworzenie eksportów payroll na podstawie zatwierdzonych wpisów czasu pracy.
- **Audyt** – rejestrowanie wybranych operacji wykonywanych na danych systemu.

5.7. Powiązanie ról z funkcjami

Administrator posiada dostęp do największej liczby funkcji i odpowiada za konfigurację systemu oraz nadzór nad danymi. Rola HR koncentruje się na danych kadrowych, takich jak pracownicy, umowy, certyfikaty, nieobecności i dokumenty. Księgowość korzysta głównie z danych związanych z czasem pracy i rozliczeniami.

Brygadzysta korzysta z danych operacyjnych, przede wszystkim z informacji o pracownikach, projektach i czasie pracy w ramach przypisanego obszaru. Pracownik jest podstawową encją systemu i może być powiązany z wieloma procesami, takimi jak przypisania do projektów, czas pracy, dokumenty, certyfikaty i nieobecności.

Czytnik RCP działa poza interfejsem użytkownika. Jego zadaniem jest przekazywanie zdarzeń technicznych do backendu. System na podstawie numeru seryjnego czytnika i identyfikatora karty RFID określa pracownika oraz projekt, którego dotyczy dane zdarzenie.

5.8. Znaczenie diagramu dla projektu

Diagram przypadków użycia porządkuje zakres funkcjonalny systemu i pokazuje, które role korzystają z poszczególnych modułów. Stanowi on punkt odniesienia dla dalszych scenariuszy użycia opisanych w kolejnym rozdziale.

Dzięki temu możliwe jest przejście od ogólnego obrazu funkcji systemu do konkretnych scenariuszy, takich jak logowanie użytkownika, zarządzanie profilem pracownika, rejestracja czasu pracy, kontrola ważności certyfikatów, obsługa nieobecności oraz przygotowanie danych rozliczeniowych.

6. Scenariusze

W niniejszym rozdziale przedstawiono scenariusze wybranych przypadków użycia systemu WorkFlow. Scenariusze opisują przebieg najważniejszych operacji z punktu widzenia użytkownika lub komponentu technicznego, a także wskazują warunki początkowe, przebieg podstawowy oraz możliwe warianty alternatywne.

Opisane scenariusze odpowiadają aktualnemu zakresowi projektu i obejmują logowanie, zarządzanie profilem pracownika, rejestrację czasu pracy z czytnika RCP, obsługę certyfikatów, nieobecności, umów, dokumentów oraz przygotowanie danych rozliczeniowych.

6.1. SC-01: Logowanie użytkownika

- **Numer:** SC-01
- **Nazwa:** Logowanie użytkownika
- **Aktor:** Użytkownik systemu
- **Stan wejścia:**
 - Użytkownik posiada konto w systemie.
 - Konto ma przypisany adres e-mail, hash hasła oraz rolę systemową.
 - Konto jest aktywne i ma włączoną możliwość logowania.
- **Przebieg:**
 1. Użytkownik otwiera formularz logowania.
 2. Wprowadza adres e-mail oraz hasło.
 3. System przesyła dane logowania do backendu.
 4. Backend wyszukuje użytkownika po adresie e-mail.
 5. Backend weryfikuje poprawność hasła.
 6. Po poprawnej weryfikacji system generuje token JWT.
 7. Token zostaje zapisany w ciasteczku sesyjnym.
 8. Użytkownik zostaje przekierowany do panelu aplikacji.
- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Użytkownik zostaje zalogowany i uzyskuje dostęp do funkcji zgodnych ze swoją rolą.
- **Błędne dane logowania:** System odrzuca próbę logowania i wyświetla komunikat o błędnych poświadczeniach.
- **Konto nieaktywne lub bez dostępu do logowania:** System nie przyznaje dostępu do aplikacji.

6.2. SC-02: Zarządzanie profilem pracownika

- **Numer:** SC-02
- **Nazwa:** Zarządzanie profilem pracownika
- **Aktor:** HR lub Administrator
- **Stan wejścia:**
 - Użytkownik jest zalogowany.
 - Użytkownik posiada rolę umożliwiającą zarządzanie danymi pracowników.
 - W systemie istnieje zakład, do którego można przypisać pracownika.
- **Przebieg:**
 1. Użytkownik przechodzi do listy pracowników.
 2. System wyświetla listę pracowników zgodną z zakresem dostępu użytkownika.
 3. Użytkownik wyszukuje pracownika lub otwiera formularz dodania nowej osoby.
 4. Użytkownik wprowadza albo modyfikuje dane podstawowe pracownika.
 5. Użytkownik określa zakład, rolę, status aktywności oraz opcjonalny identyfikator karty RFID.
 6. System waliduje poprawność danych, między innymi unikalność adresu e-mail, numeru PESEL i identyfikatora karty RFID.
 7. System zapisuje dane w bazie.
 8. Użytkownik może przejść do profilu pracownika i zarządzać powiązаныmi informacjami.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Dane pracownika zostają zapisane lub zaktualizowane.
- **Błąd walidacji:** System wyświetla komunikat i wskazuje pola wymagające poprawy.
- **Brak uprawnień:** System blokuje operację, jeżeli użytkownik nie posiada odpowiedniej roli lub dostępu do zakładu.

6.3. SC-03: Rejestracja czasu pracy przez czytnik RCP

- **Numer:** SC-03

- **Nazwa:** Rejestracja czasu pracy przez czytnik RCP

- **Aktor:** Czytnik RCP

- **Stan wejścia:**

- W systemie istnieje aktywny czytnik RCP.
- Czytnik jest przypisany do zakładu i projektu.
- Pracownik posiada przypisany identyfikator karty RFID.
- Endpoint urządzenia jest zabezpieczony tokenem technicznym.

- **Przebieg:**

1. Pracownik odbija kartę RFID na czytniku.
2. Czytnik wysyła do backendu zdarzenie zawierające numer seryjny czytnika, identyfikator karty, typ zdarzenia oraz czas zdarzenia.
3. Backend weryfikuje token urządzenia.
4. System wyszukuje aktywny czytnik na podstawie numeru seryjnego.
5. System wyszukuje aktywnego pracownika na podstawie identyfikatora karty RFID.
6. System sprawdza, czy pracownik ma dostęp do zakładu powiązanego z czytnikiem.
7. System zapisuje surowe zdarzenie w tabeli `TimeEvent`.

8. Jeżeli zdarzenie ma typ OUT, system wyszukuje wcześniejsze zdarzenie IN.
9. Na podstawie pary zdarzeń IN/OUT system tworzy wpis czasu pracy w tabeli TimeEntry.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Zdarzenie zostaje zapisane, a dla zdarzenia OUT powstaje wpis czasu pracy.
- **Nieznany czytnik:** System odrzuca zdarzenie, jeżeli numer seryjny nie odpowiada aktywnemu czytnikowi.
- **Nieznana karta RFID:** System odrzuca zdarzenie, jeżeli nie znajdzie aktywnego pracownika.
- **Brak wcześniejszego wejścia:** System zapisuje błąd logiczny i nie tworzy wpisu czasu pracy dla zdarzenia OUT.

6.4. SC-04: Weryfikacja wpisów czasu pracy

- **Numer:** SC-04

- **Nazwa:** Weryfikacja wpisów czasu pracy

- **Aktor:** HR, Administrator lub użytkownik uprawniony do weryfikacji

- **Stan wejścia:**

- W systemie istnieją wpisy czasu pracy.
- Wpisy posiadają status, na przykład PENDING.
- Użytkownik posiada uprawnienia do przeglądania i weryfikowania wpisów czasu pracy.

- **Przebieg:**

1. Użytkownik przechodzi do widoku czasu pracy.
2. System wyświetla wpisy czasu pracy z wybranego zakresu.
3. Użytkownik analizuje dane pracownika, projekt, czas rozpoczęcia, czas zakończenia oraz liczbę godzin.
4. Użytkownik zatwierdza poprawny wpis albo oznacza go jako wymagający odrzucenia.

5. System aktualizuje status wpisu czasu pracy.
6. Zatwierdzone wpisy mogą zostać wykorzystane w module rozliczeń.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Wpis uzyskuje status APPROVED i może zostać uwzględniony w rozliczeniach.
- **Odrzucenie wpisu:** Wpis uzyskuje status REJECTED i nie jest brany pod uwagę przy eksporcie payroll.
- **Brak uprawnień:** System blokuje zmianę statusu wpisu.

6.5. SC-05: Dodanie umowy pracownika

- **Numer:** SC-05
- **Nazwa:** Dodanie umowy pracownika
- **Aktor:** HR lub Administrator
- **Stan wejścia:**
 - Użytkownik jest zalogowany.
 - Użytkownik posiada dostęp do profilu pracownika.
 - Pracownik istnieje w systemie.
- **Przebieg:**
 1. Użytkownik otwiera profil pracownika.
 2. Przechodzi do sekcji umów.
 3. Wybiera opcję dodania nowej umowy.
 4. Wprowadza typ umowy, kwotę wynagrodzenia, datę rozpoczęcia oraz opcjonalną datę zakończenia.
 5. Określa, czy umowa jest aktualna.
 6. System waliduje dane formularza.
 7. System zapisuje umowę i przypisuje ją do pracownika.
- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Umowa zostaje zapisana w historii umów pracownika.
- **Brak wymaganych danych:** System wyświetla komunikat i nie zapisuje umowy.
- **Brak uprawnień:** System blokuje operację.

6.6. SC-06: Monitorowanie ważności certyfikatów

- **Numer:** SC-06
- **Nazwa:** Monitorowanie ważności certyfikatów
- **Aktor:** HR lub Administrator
- **Stan wejścia:**
 - W systemie istnieje słownik certyfikatów.
 - Pracownicy posiadają przypisane certyfikaty z datą wydania i datą ważności.
 - Użytkownik posiada dostęp do modułu certyfikatów.
- **Przebieg:**
 1. Użytkownik przechodzi do widoku certyfikatów wygasających.
 2. Wybiera zakres czasu, na przykład 30, 90 lub 365 dni.
 3. System pobiera certyfikaty, których data ważności mieści się w wybranym zakresie.
 4. System prezentuje dane pracownika, typ certyfikatu, nazwę dokumentu oraz liczbę dni do wygaśnięcia.
 5. Użytkownik może przejść do profilu pracownika i uzupełnić lub zaktualizować dane certyfikatu.
- **Zakończenie lub przebieg alternatywny:**
 - **Zakończenie pozytywne:** Użytkownik otrzymuje listę certyfikatów wymagających kontroli.
 - **Brak wyników:** System wyświetla pustą listę dla wybranego zakresu.
 - **Brak dostępu do zakładu:** System nie pokazuje danych spoza zakresu uprawnień użytkownika.

6.7. SC-07: Dodanie nieobecności pracownika

- **Numer:** SC-07
- **Nazwa:** Dodanie nieobecności pracownika
- **Aktor:** HR lub Administrator
- **Stan wejścia:**
 - Użytkownik jest zalogowany.
 - Pracownik istnieje w systemie.
 - Użytkownik posiada dostęp do profilu pracownika.
- **Przebieg:**
 1. Użytkownik otwiera profil pracownika.
 2. Przechodzi do sekcji nieobecności.
 3. Wybiera opcję dodania nowej nieobecności.
 4. Określa typ nieobecności, datę rozpoczęcia i datę zakończenia.
 5. Opcjonalnie wskazuje dokument powiązany z nieobecnością.
 6. Określa, czy nieobecność jest zatwierdzona.
 7. System zapisuje nieobecność i przypisuje ją do pracownika.
- **Zakończenie lub przebieg alternatywny:**
 - **Zakończenie pozytywne:** Nieobecność pojawia się w profilu pracownika.
 - **Niepoprawny zakres dat:** System odrzuca zapis i wyświetla komunikat.
 - **Brak uprawnień:** System blokuje operację.

6.8. SC-08: Dodanie dokumentu pracownika

- **Numer:** SC-08
- **Nazwa:** Dodanie dokumentu pracownika
- **Aktor:** HR lub Administrator
- **Stan wejścia:**

- Użytkownik posiada dostęp do profilu pracownika.
- Pracownik istnieje w systemie.
- Magazyn plików jest skonfigurowany.

- **Przebieg:**

1. Użytkownik otwiera profil pracownika.
2. Wybiera opcję dodania dokumentu.
3. Wskazuje plik dokumentu lub dane dokumentu.
4. System zapisuje plik w magazynie obiektowym albo zapisuje jego ścieżkę.
5. System tworzy rekord dokumentu w bazie danych.
6. Dokument zostaje powiązany z pracownikiem oraz opcjonalnie z umową lub certyfikatem.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Dokument zostaje zapisany i widoczny w danych pracownika.
- **Błąd zapisu pliku:** System informuje użytkownika o problemie z magazynem plików.
- **Brak danych wymaganych:** System nie tworzy rekordu dokumentu.

6.9. SC-09: Przygotowanie eksportu rozliczeniowego

- **Numer:** SC-09

- **Nazwa:** Przygotowanie eksportu rozliczeniowego

- **Aktor:** Księgowość

- **Stan wejścia:**

- Użytkownik jest zalogowany i posiada rolę księgowości.
- W systemie istnieją zatwierdzone wpisy czasu pracy.
- Wpisy nie zostały wcześniej powiązane z eksportem payroll.

- **Przebieg:**

1. Użytkownik przechodzi do modułu rozliczeń.
2. Wybiera miesiąc i rok rozliczeniowy.
3. System pobiera zatwierdzone wpisy czasu pracy z wybranego okresu.
4. System tworzy rekord eksportu rozliczeniowego.
5. System zapisuje liczbę pracowników objętych eksportem.
6. System wiąże wybrane wpisy czasu pracy z eksportem.
7. Użytkownik otrzymuje informację o przygotowaniu danych.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Eksport zostaje utworzony, a powiązane wpisy czasu pracy są oznaczone jako rozliczone w ramach danej paczki.
- **Brak zatwierdzonych wpisów:** System nie tworzy eksportu i informuje użytkownika o braku danych.
- **Dane już wyeksportowane:** System pomija wpisy powiązane z wcześniejszym eksportem.

6.10. SC-10: Uruchomienie danych demonstracyjnych

- **Numer:** SC-10
- **Nazwa:** Uruchomienie danych demonstracyjnych
- **Aktor:** Administrator techniczny
- **Stan wejścia:**
 - Środowisko Docker Compose jest uruchomione.
 - Backend ma dostęp do bazy danych.
 - W środowisku ustawiono zmienną potwierdzającą zgodę na wykonanie seeda demonstracyjnego.
- **Przebieg:**
 1. Administrator techniczny uruchamia komendę seeda demonstracyjnego.
 2. Skrypt sprawdza obecność wymaganej zmiennej środowiskowej.
 3. Skrypt czyści dane przeznaczone do prezentacji.

4. Skrypt tworzy zakłady, użytkowników, pracowników, projekty, czytelniki RCP, umowy, certyfikaty, dokumenty, nieobecności, zdarzenia czasu pracy i eksport rozliczeniowy.
5. Administrator loguje się do aplikacji na konto testowe.
6. System prezentuje komplet przykładowych danych.

- **Zakończenie lub przebieg alternatywny:**

- **Zakończenie pozytywne:** Baza danych zawiera komplet danych demonstracyjnych gotowych do prezentacji projektu.
- **Brak zmiennej zabezpieczającej:** Skrypt przerywa działanie i nie modyfikuje bazy.
- **Błąd połączenia z bazą:** Skrypt kończy się błędem i wyświetla komunikat diagnostyczny.

7. Makiety i interfejs użytkownika

Interfejs użytkownika systemu WorkFlow został zaprojektowany z myślą o pracy administracyjnej wykonywanej z poziomu przeglądarki internetowej. Głównym założeniem było przygotowanie widoków, które pozwalają szybko przejść do najważniejszych modułów systemu: pracowników, zakładów, projektów, czasu pracy, certyfikatów, dokumentów, nieobecności oraz rozliczeń.

W początkowej fazie projektu przygotowano makiety interfejsu, które pozwoliły określić ogólny układ aplikacji oraz rozmieszczenie najważniejszych informacji. W późniejszej implementacji część widoków została dostosowana do aktualnego zakresu funkcjonalnego systemu oraz do zastosowanego stosu technologicznego.

7.1. Założenia projektowe interfejsu

Podczas projektowania interfejsu przyjęto następujące założenia:

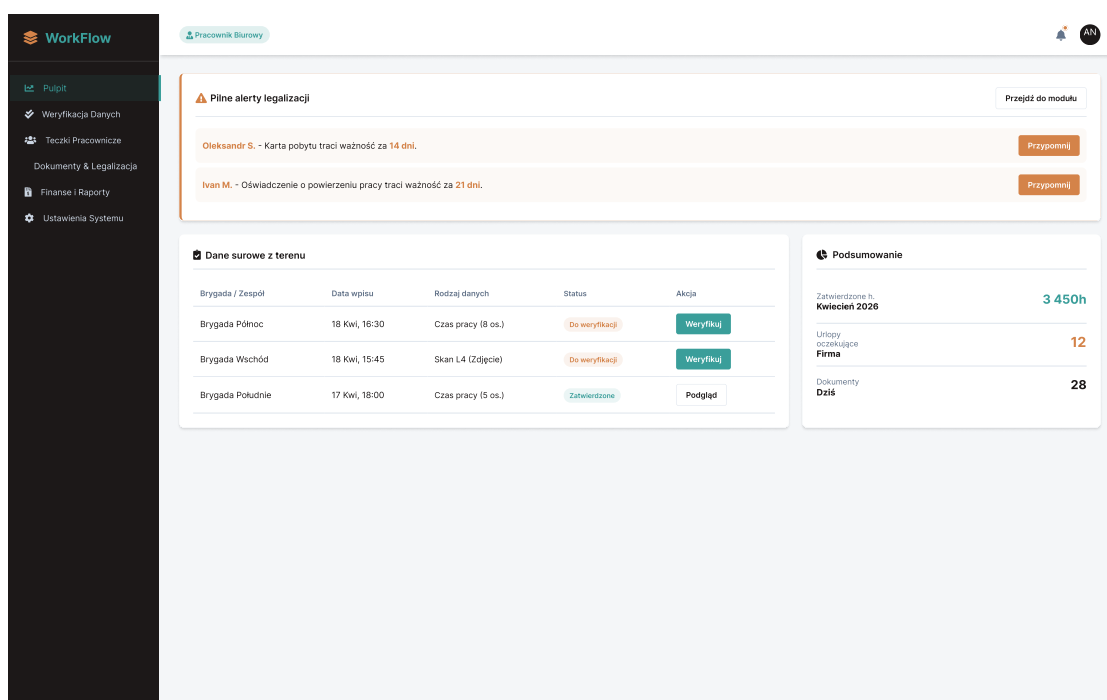
- system powinien posiadać czytelną nawigację boczną,
- główne moduły powinny być dostępne z poziomu panelu użytkownika,
- widoki powinny być spójne pod względem układu formularzy, tabel i przycisków,

- dane powinny być możliwe do filtrowania i wyszukiwania,
- formularze powinny wskazywać wymagane dane oraz obsługiwać błędy walidacji,
- interfejs powinien poprawnie działać w trybie jasnym i ciemnym,
- użytkownik powinien widzieć tylko te elementy, do których ma dostęp zgodnie ze swoją rolą.

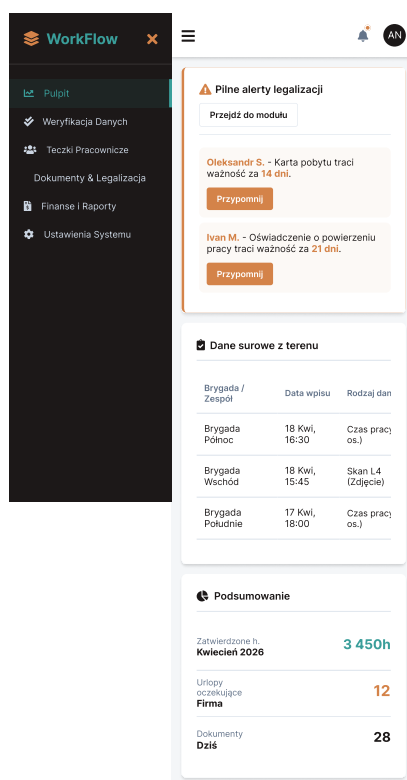
Aplikacja ma charakter panelu administracyjnego. Z tego względu najważniejsza była czytelność danych, łatwość przeglądania tabel oraz możliwość szybkiego przejścia do szczegółów rekordu.

7.2. Pulpit główny

Pulpit główny stanowi pierwszy ekran po zalogowaniu użytkownika. Jego zadaniem jest zebranie najważniejszych informacji oraz zapewnienie szybkiego dostępu do modułów systemu.



Rys. 7.1: Widok desktopowy – pulpit główny systemu WorkFlow



Rys. 7.2: Widok mobilny – pulpit główny systemu WorkFlow

Najważniejsze elementy pulpitu:

- **Nawigacja do modułów** – użytkownik może przejść do widoków pracowników, zakładów, projektów, czasu pracy, certyfikatów i rozliczeń.
- **Kontekst zakładu** – system uwzględnia zakład aktywny dla użytkownika, co pozwala ograniczyć zakres prezentowanych danych.
- **Podsumowania** – widok może prezentować skrócone informacje dotyczące pracowników, projektów, certyfikatów lub czasu pracy.
- **Dostęp zależny od roli** – zakres widocznych elementów zależy od tego, czy użytkownik pełni rolę administratora, HR, księgowości, brygadzysty lub inną rolę systemową.

Pulpit główny nie jest samodzielnym modułem biznesowym. Pełni rolę punktu startowego, który pozwala użytkownikowi przejść do dalszych obszarów systemu.

7.3. Profil pracownika

Profil pracownika jest jednym z najważniejszych widoków systemu. Łączy w jednym miejscu dane podstawowe pracownika oraz informacje powiązane z innymi modułami.

Profil Pracownika

Oleksandr Stepanenko
Monter Konstrukcji Stalowych | Brygada Północ
Aktywny (Umowa Zlecenie)

Aktualna Stawka
34.50 PLN / h

[Drukuj Kartę](#) [Edytuj Dane](#)

Ogólne & Legalizacja | Dokumenty (Scany) | Szkolenia & BHP | Statystyki Pracy

Dane podstawowe

ID Pracownika:	#EMP-089
Data urodzenia:	12.05.1992
PESEL / Paszport:	FB 9928374
Zatrudnienie od:	01.02.2024
Telefon:	+48 777 000 111

Legalizacja pracy

Karta Pobytu:	TAK
Ważna do:	15.11.2024
Oświadczenie:	Wygasiło!

Uprawnienia i Szkolenia

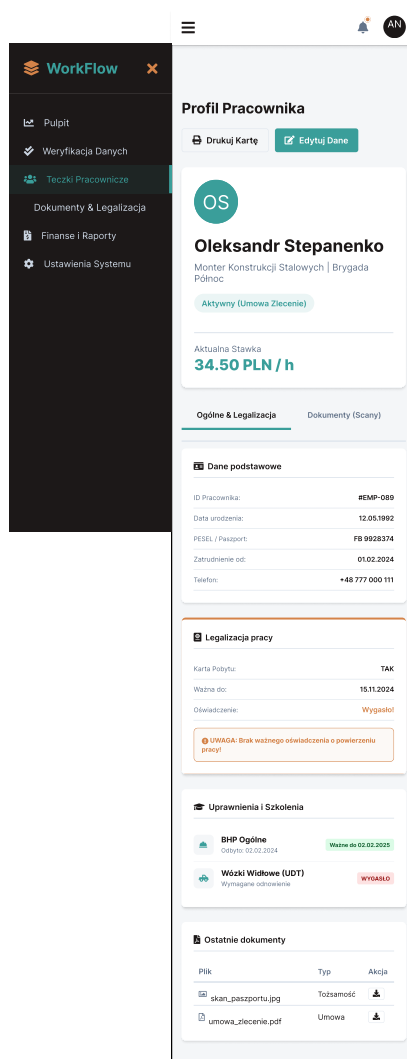
- BHP Ogólne**
Odbity: 02.02.2024 Ważne do 02.02.2025
- Wózki Widłowe (UDT)**
Wymagane odnowienie WYGASŁO

Ostatnie dokumenty

Plik	Typ	Akcja
skan_paszportu.jpg	Tożsamość	
umowa_zlecenie.pdf	Umowa	

UWAGA: Brak ważnego oświadczenia o powierzeniu pracy!

Rys. 7.3: Widok desktopowy – profil pracownika



Rys. 7.4: Widok mobilny – profil pracownika

Zakres danych prezentowanych w profilu pracownika:

- dane podstawowe, takie jak imię, nazwisko, PESEL, e-mail, rola i status aktywności,
- przypisanie do zakładu,
- identyfikator karty RFID,
- umowy pracownika,
- certyfikaty, badania i szkolenia,
- dokumenty powiązane z pracownikiem,

- nieobecności,
- przypisania do projektów,
- wpisy czasu pracy.

Profil pracownika pełni funkcję centralnego widoku kadrowego. Dzięki temu użytkownik nie musi szukać danych pracownika w wielu niezależnych miejscach. Powiązane informacje są prezentowane w jednym kontekście, co ułatwia obsługę danych kadrowych i organizacyjnych.

7.4. Widoki tabelaryczne

Znaczna część interfejsu systemu opiera się na widokach tabelarycznych. Dotyczy to między innymi listy pracowników, zakładów, projektów, certyfikatów, czasu pracy oraz widoków rozliczeniowych.

Widoki tabelaryczne powinny umożliwiać:

- szybkie przeglądanie większej liczby rekordów,
- wyszukiwanie danych,
- filtrowanie według wybranych kryteriów,
- przejście do szczegółów rekordu,
- prezentację statusów w czytelnej formie,
- zachowanie spójnego układu kolumn i akcji.

W przypadku listy pracowników szczególne znaczenie mają filtry po imieniu, nazwisku, adresie e-mail, numerze PESEL, roli oraz statusie. Dla certyfikatów istotne są natomiast typ, nazwa, data ważności oraz liczba dni pozostałych do wygaśnięcia.

7.5. Formularze

System wykorzystuje formularze do dodawania i edycji danych. Formularze występują między innymi w modułach pracowników, zakładów, projektów, umów, certyfikatów i nieobecności.

Podczas projektowania formularzy przyjęto, że:

- pola powinny być opisane w sposób jednoznaczny,
- dane wymagane powinny być walidowane przed zapisem,
- błędy powinny być prezentowane użytkownikowi w czytelny sposób,
- formularze powinny być możliwie krótkie i dostosowane do kontekstu,
- po zapisie danych użytkownik powinien otrzymać informację o rezultacie operacji.

Przykładem takiego formularza jest edycja danych pracownika, gdzie użytkownik może zaktualizować dane osobowe, adres e-mail, numer PESEL, identyfikator karty RFID, rolę oraz status aktywności.

7.6. Tryb jasny i ciemny

W systemie przewidziano obsługę trybu jasnego i ciemnego. Wymagało to ujednolicenia kolorów tła, tekstu, obramowań, pól formularzy oraz elementów tabelarycznych.

Obsługa dwóch motywów ma znaczenie praktyczne, ponieważ aplikacja może być używana w różnych warunkach oświetleniowych. Dodatkowo wymusza ona bardziej konsekwentne stosowanie tokenów kolorystycznych zamiast sztywnych kolorów przypisanych wyłącznie do jednego motywu.

7.7. Znaczenie makiet dla implementacji

Makiety stanowiły punkt wyjścia do określenia sposobu prezentacji danych oraz układu najważniejszych widoków. W trakcie implementacji część założeń została zmodyfikowana, aby lepiej dopasować interfejs do aktualnego modelu danych i dostępnych modułów.

Najważniejsze decyzje wynikające z pracy nad interfejsem to:

- wykorzystanie bocznej nawigacji jako głównego sposobu poruszania się po aplikacji,
- centralne znaczenie profilu pracownika,
- prezentowanie powiązanych danych w sekcjach,
- stosowanie tabel dla list rekordów,
- oddzielenie widoków operacyjnych od widoków administracyjnych,

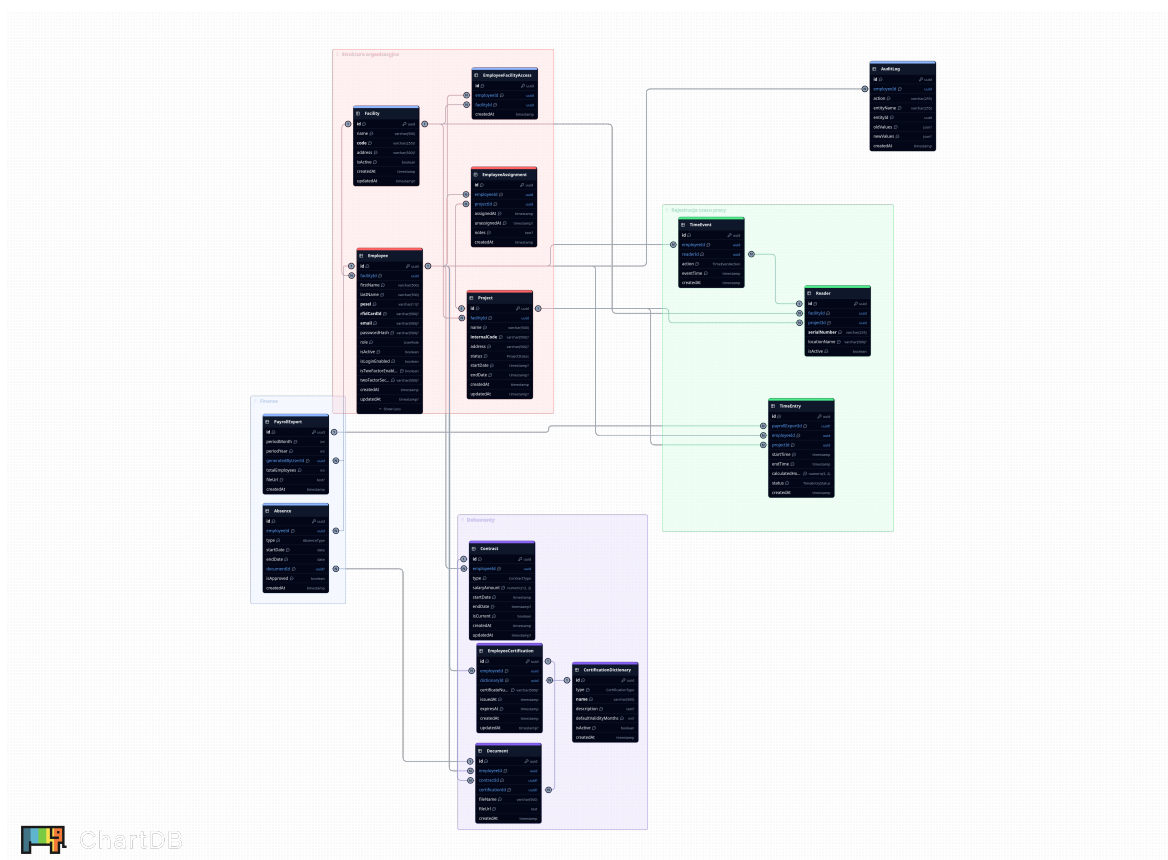
- zachowanie spójności wyglądu pomiędzy trybem jasnym i ciemnym.

W rezultacie interfejs użytkownika został podporządkowany strukturze danych oraz najważniejszym procesom systemu.

8. Model danych i diagram ERD

Model danych stanowi podstawę działania systemu WorkFlow. To w bazie danych przechowywane są informacje o pracownikach, zakładach, projektach, umowach, certyfikatach, dokumentach, nieobecnościach, zdarzeniach RCP, wpisach czasu pracy oraz eksportach rozliczeniowych. Poprawne zaprojektowanie relacji pomiędzy tymi danymi ma bezpośredni wpływ na spójność systemu oraz możliwość dalszego rozwoju aplikacji.

W projekcie wykorzystano relacyjną bazę danych PostgreSQL oraz narzędzie Prisma ORM. Schemat bazy został zdefiniowany w pliku `schema.prisma`, natomiast graficzna reprezentacja modelu została przygotowana w postaci diagramu ERD.



Rys. 8.1: Logiczny model danych systemu WorkFlow

8.1. Założenia modelu danych

Podczas projektowania modelu danych przyjęto kilka podstawowych założeń. Pierwszym z nich było zastosowanie identyfikatorów typu `UUID` jako kluczy głównych. Rozwiązanie to pozwala na jednoznaczne identyfikowanie rekordów i jest wygodne w systemach, które mogą być rozwijane o dodatkowe integracje lub wiele środowisk uruchomieniowych.

Drugim założeniem było rozdzielenie danych źródłowych od danych przetworzonych. Najlepiej widać to w module czasu pracy. Tabela `TimeEvent` przechowuje surowe zdarzenia pochodzące z czytników RCP, natomiast tabela `TimeEntry` przechowuje przetworzone wpisy czasu pracy, które mogą być dalej weryfikowane i wykorzystywane w rozliczeniach.

Trzecim założeniem było powiązanie danych z zakładami. Pracownicy, projekty i czytniki RCP są przypisane do zakładu, co pozwala ograniczać zakres widocznych danych w zależności od uprawnień użytkownika.

Model danych został ograniczony do modułów obecnych w aktualnym zakresie projektu. Z tego względu w dokumentacji i diagramie nie uwzględniono modułu sprzętu, ponieważ został on wyłączony z zakresu realizowanej aplikacji.

8.2. Podział modelu na obszary

Model danych można podzielić na pięć głównych obszarów logicznych:

- **Struktura organizacyjna** – obejmuje zakłady, pracowników, dostęp do zakładów, projekty oraz przypisania pracowników do projektów.
- **Rejestracja czasu pracy** – obejmuje czytniki RCP, surowe zdarzenia wejścia i wyjścia oraz przetworzone wpisy czasu pracy.
- **Kadry i dokumentacja** – obejmuje umowy, dokumenty, certyfikaty, słownik certyfikatów oraz nieobecności.
- **Rozliczenia** – obejmuje eksporty payroll oraz powiązanie zatwierdzonych wpisów czasu pracy z paczką rozliczeniową.
- **Audyt** – obejmuje rejestrowanie wybranych operacji wykonywanych na danych systemu.

Taki podział ułatwia analizę modelu oraz pokazuje, które tabele pełnią funkcję centralną, a które są tabelami pomocniczymi lub relacyjnymi.

8.3. Słownik głównych encji

W tabeli 8.1 przedstawiono najważniejsze encje występujące w modelu danych systemu WorkFlow.

Encja	Znaczenie w systemie
Facility	Zakład lub jednostka organizacyjna. Do zakładu przypisywani są pracownicy, projekty oraz czytniki RCP.
Employee	Centralna tabela osób. Przechowuje dane pracownika oraz dane konta użytkownika, takie jak e-mail, rola, status aktywności i identyfikator karty RFID.
EmployeeFacilityAccess	Tabela określająca, do których zakładów użytkownik posiada dostęp. Umożliwia obsługę wielu zakładów przez jednego użytkownika.
Project	Projekt, budowa lub miejsce pracy. Jest przypisany do zakładu i może posiadać przypisanych pracowników oraz czytniki RCP.
EmployeeAssignment	Historia przypisań pracowników do projektów. Pozwala określić, kto, od kiedy i do jakiego projektu został przypisany.
Reader	Czytnik RCP. Jest powiązany z zakładem oraz projektem i służy do rejestrowania zdarzeń czasu pracy.
TimeEvent	Surowe zdarzenie z czytnika RCP. Przechowuje informację o pracowniku, czytniku, typie zdarzenia oraz czasie odbicia.
TimeEntry	Przetworzony wpis czasu pracy utworzony na podstawie zdarzeń IN i OUT. Zawiera czas rozpoczęcia, zakończenia, liczbę godzin i status.
Contract	Umowa pracownika. Przechowuje typ umowy, wynagrodzenie, daty obowiązywania oraz informację, czy umowa jest aktualna.
CertificationDictionary	Słownik typów certyfikatów, szkoleń, badań i uprawnień.
EmployeeCertification	Konkretny certyfikat przypisany do pracownika, wraz z datą wydania i datą ważności.
Absence	Nieobecność pracownika, na przykład urlop, L4, nieobecność nieusprawiedliwiona lub specjalna.
Document	Metadane dokumentu pracownika. Plik przechowywany jest w MinIO, a baza przechowuje jego nazwę, lokalizację i powiązania.
PayrollExport	Eksport rozliczeniowy obejmujący zatwierdzone wpisy czasu pracy dla danego okresu.
PrintTemplate	Printout szablonu dokumentu, który może być używany do drukowania.

8.4. Struktura organizacyjna

Podstawą modelu organizacyjnego jest tabela **Facility**. Reprezentuje ona zakład lub jednostkę organizacyjną. Z zakładem powiązani są pracownicy, projekty oraz czytniki RCP. Dzięki temu dane można grupować według lokalizacji lub oddziałów.

Tabela **Employee** pełni podwójną rolę. Z jednej strony przechowuje dane pracownika, takie jak imię, nazwisko, PESEL, status aktywności i identyfikator karty RFID. Z drugiej strony zawiera pola potrzebne do logowania, takie jak adres e-mail, hash hasła, rola systemowa oraz flaga określająca, czy logowanie jest włączone.

Dostęp użytkowników do zakładów został opisany w tabeli **EmployeeFacilityAccess**. Dzięki temu użytkownik może posiadać dostęp do więcej niż jednego zakładu. Jest to istotne szczególnie dla administratora, HR lub księgowości, którzy mogą obsługiwać dane wielu jednostek.

Relacja pomiędzy pracownikiem a projektem została zrealizowana przez tabelę **EmployeeAssignment**. Tabela ta umożliwia przechowywanie historii przypisań, a nie tylko aktualnego stanu. Dzięki temu można określić, w jakim okresie pracownik był związany z danym projektem.

8.5. Rejestracja czasu pracy

Moduł czasu pracy składa się z trzech głównych tabel: **Reader**, **TimeEvent** oraz **TimeEntry**.

Tabela **Reader** przechowuje informacje o czytniku RCP. Czytnik posiada numer seryjny, nazwę lokalizacji oraz powiązanie z zakładem i projektem. Oznacza to, że zdarzenie odbicia karty na czytniku można powiązać nie tylko z pracownikiem, ale również z miejscem pracy.

Tabela **TimeEvent** przechowuje surowe zdarzenia z czytników. Każde zdarzenie zawiera pracownika, czytnik, typ zdarzenia oraz czas zdarzenia. Typ zdarzenia jest ograniczony do dwóch wartości: **IN** oraz **OUT**.

Tabela **TimeEntry** przechowuje dane przetworzone. Wpis czasu pracy powstaje na podstawie pary zdarzeń wejścia i wyjścia. Zawiera on czas rozpoczęcia, czas zakończenia, wyliczoną liczbę godzin, projekt oraz status wpisu.

Pole	Typ	Opis
id	UUID	Klucz główny wpisu czasu pracy.
payrollExportId	UUID, opcjonalny	Powiązanie z eksportem rozliczeniowym, jeżeli wpis został już ujęty w paczce payroll.
employeeId	UUID	Pracownik, którego dotyczy wpis czasu pracy.
projectId	UUID	Projekt, na którym pracownik wykonywał pracę.
startTime	timestamp	Czas rozpoczęcia pracy.
endTime	timestamp	Czas zakończenia pracy.
calculatedHours	numeric(5,2)	Wyliczona liczba przepracowanych godzin.
status	TimeEntryStatus	Status wpisu czasu pracy: PENDING, APPROVED lub REJECTED.
createdAt	timestamp	Data utworzenia wpisu.

Tab. 8.2: Struktura tabeli TimeEntry

Rozdzielenie `TimeEvent` i `TimeEntry` jest ważne projektowo. Pozwala zachować pierwotne dane z urządzenia, a jednocześnie tworzyć dane robocze, które mogą być zatwierdzane, odrzucane lub eksportowane do rozliczeń.

8.6. Kadry i dokumentacja

Dane kadrowe zostały rozbite na kilka tabel, aby uniknąć przechowywania wszystkich informacji bezpośrednio w tabeli pracownika. Takie podejście poprawia czytelność modelu i umożliwia przechowywanie historii.

Tabela `Contract` przechowuje umowy pracownika. Każda umowa posiada typ, kwotę wynagrodzenia, datę rozpoczęcia, opcjonalną datę zakończenia oraz informację, czy jest aktualna. Pozwala to przechowywać zarówno bieżące, jak i historyczne warunki zatrudnienia.

Tabele `CertificationDictionary` i `EmployeeCertification` obsługują certyfikaty, badania i szkolenia. Pierwsza tabela definiuje słownik możliwych certyfikatów,

natomiast druga przechowuje konkretne certyfikaty przypisane do pracowników. Dzięki temu można kontrolować daty ważności dokumentów.

Tabela **Absence** przechowuje nieobecności pracowników. Zawiera typ nieobecności, zakres dat, status zatwierdzenia oraz opcjonalne powiązanie z dokumentem.

Tabela **Document** przechowuje metadane dokumentów. Sam plik nie jest zapisywany bezpośrednio w bazie danych. W bazie przechowywana jest nazwa pliku, adres lub ścieżka do pliku oraz powiązanie z pracownikiem, umową lub certyfikatem.

8.7. Rozliczenia

Tabela **PayrollExport** reprezentuje paczkę danych rozliczeniowych dla danego miesiąca i roku. Przechowuje informację o użytkowniku, który wygenerował eksport, liczbie pracowników objętych eksportem oraz opcjonalnej ścieżce do pliku eksportu.

Z tabelą **PayrollExport** powiązana jest tabela **TimeEntry**. Jeżeli pole **payrollExportId** w tabeli **TimeEntry** jest wypełnione, oznacza to, że dany wpis czasu pracy został ujęty w konkretnej paczce rozliczeniowej. Takie rozwiązanie ogranicza ryzyko ponownego rozliczenia tych samych godzin.

8.8. Audit log

Tabela **AuditLog** została przewidziana do rejestrowania wybranych operacji wykonywanych na danych systemu. Każdy wpis audytowy zawiera użytkownika wykonującego operację, nazwę akcji, nazwę encji, identyfikator rekordu oraz opcjonalnie wartości przed zmianą i po zmianie.

Zastosowanie pól typu **Json** pozwala przechowywać zmiany w elastycznej formie, bez konieczności tworzenia osobnych tabel dla każdego rodzaju operacji. Mechanizm ten jest szczególnie przydatny przy analizie zmian dotyczących danych kadrowych, umów lub wpisów czasu pracy.

8.9. Typy wyliczeniowe

W modelu danych zastosowano kilka typów wyliczeniowych, które ograniczają możliwe wartości wybranych pól. Dzięki temu baza danych nie przyjmuje przypadkowych wartości tekstowych, a aplikacja może opierać się na stałym zbiorze statusów i kategorii.

Najważniejsze typy wyliczeniowe to:

- `UserRole` – role użytkowników: ADMIN, HR, OFFICE, FOREMAN, ACCOUNTING, WORKER,
- `ProjectStatus` – statusy projektów: PLANNED, ACTIVE, COMPLETED, SUSPENDED,
- `TimeEventAction` – typy zdarzeń RCP: IN, OUT,
- `TimeEntryStatus` – statusy wpisów czasu pracy: PENDING, APPROVED, REJECTED,
- `ContractType` – typy umów: UOP, UZ, UD, B2B,
- `CertificationType` – typy certyfikatów: BHP, MEDICAL, UDT, OTHER,
- `AbsenceType` – typy nieobecności: HOLIDAY, SICK_LEAVE, UNEXCUSED, SPECIAL.

8.10. Implementacja fizyczna bazy danych

Fizyczna struktura bazy danych jest generowana na podstawie modeli Prisma. Przykładowa definicja tabeli `TimeEntry` w schemacie Prisma wygląda następująco:

Listing 1: Fragment modelu `TimeEntry` w schemacie Prisma

```

1 model TimeEntry {
2   id          String          @id @default(dbgenerated("
      gen_random_uuid()")) @db.Uuid
3   payrollExportId String?      @db.Uuid
4   employeeId   String          @db.Uuid
5   projectId    String          @db.Uuid
6   startTime    DateTime        @db.Timestamp(6)
7   endTime      DateTime        @db.Timestamp(6)
8   calculatedHours Decimal       @db.Decimal(5, 2)
9   status        TimeEntryStatus @default(PENDING)
10  createdAt     DateTime        @default(now()) @db.Timestamp(6)
11
12  payrollExport  PayrollExport? @relation(fields: [payrollExportId],
      references: [id])
13  employee       Employee        @relation(fields: [employeeId],
      references: [id])
14  project        Project         @relation(fields: [projectId],
      references: [id])
15 }
```

W praktyce struktura bazy może zostać zsynchronizowana z modelem Prisma za pomocą polecenia:

Listing 2: Synchronizacja bazy danych ze schematem Prisma

```
1 docker compose -f docker-compose.prod.yml --env-file .env exec backend  
  npx prisma db push --config prisma.config.ts
```

Takie podejście pozwala utrzymać jedno źródło prawdy dla struktury bazy danych w kodzie projektu.

8.11. Strategia dostępu do danych

Backend komunikuje się z bazą danych za pomocą Prisma ORM. Prisma odpowiada za mapowanie modeli aplikacji na tabele bazy danych, wykonywanie zapytań oraz obsługę relacji pomiędzy encjami.

W projekcie większość operacji może być realizowana przez standardowe zapytania ORM. Dotyczy to między innymi dodawania pracowników, edycji danych, pobierania list, tworzenia umów, certyfikatów, nieobecności oraz wpisów czasu pracy.

W przypadku bardziej złożonych raportów lub eksportów możliwe jest wykorzystanie zapytań agregujących. Aktualna wersja projektu skupia się jednak przede wszystkim na czytelnym modelu relacyjnym i poprawnym powiązaniu danych, a nie na zaawansowanej analizie SQL.

8.12. Znaczenie modelu danych dla systemu

Model danych systemu **WorkFlow** został zaprojektowany tak, aby odwzorować podstawowe zależności występujące w procesach kadrowych i ewidencji czasu pracy. Najważniejszą encją jest pracownik, który łączy się z większością pozostałych obszarów systemu: zakładami, projektami, dokumentami, certyfikatami, umowami, nieobecnościami i czasem pracy.

Dzięki relacyjnemu modelowi danych system może zachować spójność pomiędzy modułami. Przykładowo wpis czasu pracy nie istnieje samodzielnie, lecz jest powiązany z pracownikiem i projektem. Dokument nie jest anonimowym plikiem, lecz posiada właściciela i może być powiązany z umową lub certyfikatem. Certyfikat nie jest tylko tekstową informacją, ale wynika ze słownika i posiada datę ważności.

Takie ustrukturyzowanie danych umożliwia dalszy rozwój systemu, między innymi o dodatkowe raporty, powiadomienia, integracje księgowe lub bardziej rozbudowane mechanizmy kontroli dostępu.

9. Wykorzystanie narzędzi AI w procesie realizacji projektu

W trakcie realizacji projektu WorkFlow autor korzystał z narzędzi opartych na sztucznej inteligencji jako wsparcia w analizie problemów, porządkowaniu zadań, redagowaniu dokumentacji oraz weryfikacji wybranych rozwiązań technicznych. Wykorzystywane były przede wszystkim narzędzia ChatGPT oraz Gemini, przy czym ChatGPT był używany częściej ze względu na lepsze dopasowanie odpowiedzi do kontekstu projektu i możliwość prowadzenia dłuższej, spójnej pracy nad kolejnymi etapami systemu.

Narzędzia AI nie zastępowały samodzielnej realizacji projektu. Pełniły funkcję konsultacyjną, podobną do korzystania z dokumentacji technicznej, wyszukiwarki, forów programistycznych lub narzędzi wspierających analizę kodu. Ostateczne decyzje projektowe, implementacyjne i redakcyjne były podejmowane przez autora projektu, a zaproponowane rozwiązania były każdorazowo weryfikowane w praktyce.

9.1. Zakres wykorzystania narzędzi AI

Narzędzia AI były wykorzystywane głównie w tych obszarach, w których przyspieszały analizę lub pomagały uporządkować pracę. Dotyczyło to zarówno części programistycznej, jak i dokumentacyjnej projektu.

Najważniejsze zastosowania obejmowały:

- analizę błędów pojawiających się podczas pracy nad frontendem i backendem,
- interpretację komunikatów TypeScript, ESLint, Prisma, Docker oraz Node.js,
- porządkowanie kolejności zadań wykonywanych w projekcie,
- przygotowywanie propozycji komend testowych i diagnostycznych,
- analizę problemów z uruchomieniem aplikacji w kontenerach Docker,
- wsparcie przy konfiguracji środowiska produkcyjnego,
- pomoc w przygotowaniu danych demonstracyjnych,
- redagowanie i aktualizację dokumentacji technicznej,
- upraszczanie zbyt rozbudowanych lub nieprecyzyjnych opisów.

Szczególnie istotne było wykorzystanie AI jako narzędzia do bieżącej analizy problemów. W przypadku pojawienia się błędu autor mógł przedstawić komunikat z terminala lub fragment kodu, a następnie otrzymać propozycję przyczyny problemu oraz możliwych kroków diagnostycznych.

9.2. Wsparcie podczas implementacji

Podczas implementacji systemu narzędzia AI były wykorzystywane do analizy i wyjaśniania problemów technicznych. Dotyczyło to między innymi błędów kompilacji TypeScript, ostrzeżeń ESLint, problemów z działaniem formularzy, konfiguracją zmiennych środowiskowych oraz uruchomieniem aplikacji w Dockerze.

Przykładowe obszary, w których AI wspierało pracę implementacyjną, to:

- analiza błędów związanych z typami danych w TypeScript,
- poprawianie problemów wykrywanych przez ESLint,
- diagnozowanie problemów z logowaniem użytkownika,
- analiza konfiguracji frontendowego adresu API,
- pomoc przy konfiguracji Dockerfile dla frontendu i backendu,
- analiza błędów uruchamiania aplikacji w kontenerach,
- wsparcie przy pracy z Prisma i inicjalizacji bazy danych,
- przygotowanie oraz weryfikacja seedów bazy danych.

W praktyce AI nie było traktowane jako źródło gotowych, bezwarunkowo poprawnych rozwiązań. Każda propozycja wymagała sprawdzenia w kodzie, uruchomienia aplikacji oraz oceny, czy pasuje do aktualnej struktury projektu. W wielu przypadkach wygenerowane sugestie wymagały modyfikacji, uproszczenia albo dostosowania do istniejących plików i konwencji.

9.3. Wsparcie przy konfiguracji środowiska

Istotnym elementem projektu było przygotowanie środowiska uruchomieniowego opartego na Docker Compose. W tym obszarze narzędzia AI pomagały w analizie zależności pomiędzy kontenerami, sposobie przekazywania zmiennych środowiskowych oraz

diagnozowaniu błędów wynikających z różnic pomiędzy środowiskiem lokalnym a kontenerowym.

AI wspierało między innymi analizę problemów związanych z:

- brakiem zbudowanych plików aplikacji w katalogu `dist`,
- błędną ścieżką startową backendu,
- brakiem wymaganych zmiennych środowiskowych w kontenerze,
- konfiguracją połączenia Prisma z bazą PostgreSQL,
- przekierowaniem ruchu przez Nginx,
- różnicą pomiędzy adresem API widocznym dla przeglądarki a adresem API widocznym wewnątrz sieci Docker.

W tym zakresie AI pełniło rolę pomocniczą przy interpretacji logów i wskazywaniu kolejnych komend diagnostycznych. Ostateczna weryfikacja odbywała się poprzez uruchomienie kontenerów, analizę logów, testowanie endpointów za pomocą `curl` oraz sprawdzenie działania aplikacji w przeglądarce.

9.4. Wsparcie przy danych demonstracyjnych

Narzędzia AI zostały również wykorzystane podczas przygotowywania danych demonstracyjnych. Dane te były potrzebne do prezentacji działania systemu bez konieczności ręcznego wprowadzania dużej liczby rekordów.

Wsparcie AI obejmowało:

- zaplanowanie zakresu danych demonstracyjnych,
- przygotowanie struktury seeda prezentacyjnego,
- analizę błędów TypeScript w skrypcie seedującym,
- uporządkowanie danych testowych tak, aby obejmowały najważniejsze moduły systemu,
- dodanie zabezpieczenia przed przypadkowym uruchomieniem destrukcyjnego seeda.

Dzięki temu możliwe było przygotowanie danych obejmujących użytkowników, zakłady, pracowników, projekty, czytniki RCP, umowy, certyfikaty, dokumenty, nieobecności, wpisy czasu pracy oraz przykładowy eksport rozliczeniowy. Dane te zostały następnie sprawdzone w działającej aplikacji.

9.5. Wsparcie przy dokumentacji

Drugim ważnym obszarem wykorzystania narzędzi AI była dokumentacja projektu. W tym przypadku AI pomagało przede wszystkim w porządkowaniu treści, aktualizacji opisów do aktualnego stanu aplikacji oraz usuwaniu zbyt ogólnych lub przesadnie marketingowych sformułowań.

W dokumentacji AI wspierało:

- redakcję opisu celu projektu,
- uporządkowanie wymagań funkcjonalnych i нефunkcyjnych,
- aktualizację scenariuszy użycia,
- opis modelu danych i diagramu ERD,
- opis stosu technologicznego,
- przygotowanie instrukcji prezentacji projektu,
- ujednolicenie stylu językowego dokumentacji.

Ważnym założeniem było zachowanie formalnego charakteru dokumentacji bez nadmiernego rozbudowywania opisów. Z tego powodu część propozycji była upraszczana lub odrzucana, jeżeli brzmiała zbyt marketingowo albo sugerowała funkcje, które nie zostały zaimplementowane w aktualnej wersji projektu.

9.6. Weryfikacja odpowiedzi generowanych przez AI

Korzystanie z narzędzi AI wymagało zachowania ostrożności. Odpowiedzi generowane przez modele językowe mogą być niepełne, nieaktualne albo niedopasowane do konkretnego projektu. Z tego względu żadna propozycja nie była traktowana jako automatycznie poprawna.

Weryfikacja odbywała się poprzez:

- analizę istniejącego kodu źródłowego,
- uruchamianie aplikacji lokalnie,
- sprawdzanie logów backendu, frontendu i kontenerów,
- testowanie endpointów za pomocą zapytań HTTP,
- kontrolę działania funkcji w interfejsie użytkownika,
- sprawdzanie zmian w systemie kontroli wersji Git,
- porównywanie dokumentacji z rzeczywistym zakresem aplikacji.

Taki sposób pracy ograniczał ryzyko bezrefleksyjnego przyjęcia błędnych sugestii. AI było wykorzystywane jako narzędzie wspomagające analizę i organizację pracy, a nie jako samodzielny wykonawca projektu.

9.7. Porównanie wykorzystanych narzędzi

W trakcie pracy wykorzystywane były dwa narzędzia: Gemini oraz ChatGPT. Oba rozwiązania mogły wspierać analizę problemów i redakcję treści, jednak w praktyce większe znaczenie w projekcie miało narzędzie ChatGPT.

Gemini było wykorzystywane pomocniczo, głównie do porównywania propozycji odpowiedzi lub uzyskania alternatywnego spojrzenia na dany problem. ChatGPT okazał się bardziej przydatny podczas dłuższej pracy nad projektem, ponieważ lepiej utrzymywał kontekst kolejnych etapów, takich jak rozwój modułów, poprawki błędów, konfiguracja Docker oraz aktualizacja dokumentacji.

Nie oznacza to jednak, że odpowiedzi ChatGPT były przyjmowane bez weryfikacji. Narzędzie to było traktowane jako asystent konsultacyjny, którego propozycje należało sprawdzić i dostosować do rzeczywistego kodu projektu.

9.8. Wpływ AI na przebieg pracy

Wykorzystanie narzędzi AI wpłynęło przede wszystkim na tempo pracy i sposób organizacji zadań. Dzięki możliwości szybkiej analizy błędów i propozycji kolejnych kroków łatwiej było przechodzić przez problemy techniczne oraz utrzymywać porządek w realizowanych zadaniach.

Największe korzyści z wykorzystania AI to:

- szybsza analiza błędów i komunikatów diagnostycznych,
- łatwiejsze planowanie kolejnych etapów pracy,
- możliwość szybkiego porównania kilku wariantów rozwiązania,
- wsparcie przy redakcji dokumentacji,
- większa konsekwencja w nazewnictwie i opisie modułów,
- pomoc w przygotowaniu komend testowych i diagnostycznych.

Jednocześnie wykorzystanie AI nie zwalniało autora z odpowiedzialności za projekt. Kod musiał zostać uruchomiony, przetestowany i dopasowany do istniejącej struktury aplikacji. Dokumentacja musiała zostać porównana z rzeczywistym zakresem systemu, aby nie opisywała funkcji, które nie są częścią projektu.

9.9. Podsumowanie

Narzędzia AI były istotnym wsparciem podczas realizacji projektu **WorkFlow**, szczególnie w zakresie analizy błędów, organizacji pracy i redakcji dokumentacji. Ich rola miała jednak charakter pomocniczy. Projekt, jego zakres, decyzje techniczne, testowanie oraz ostateczny kształt dokumentacji pozostawały pod kontrolą autora.

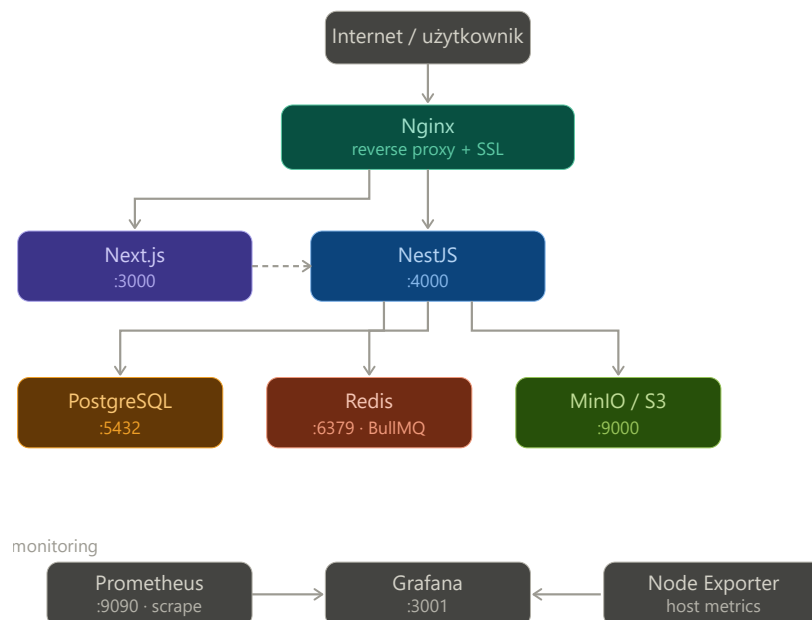
Takie wykorzystanie AI można traktować jako element współczesnego procesu pracy programistycznej. Narzędzia te pomagają szybciej analizować problemy i porządkować informacje, ale wymagają krytycznego podejścia, weryfikacji oraz świadomego dostosowania wygenerowanych sugestii do rzeczywistego projektu.

10. Stos technologiczny

Stos technologiczny systemu **WorkFlow** został dobrany w taki sposób, aby umożliwić stworzenie aplikacji webowej z wyraźnym podziałem na frontend, backend, bazę danych, magazyn plików oraz warstwę uruchomieniową. Przy wyborze technologii uwzględniono możliwość pracy w języku TypeScript, dostępność narzędzi wspierających szybkie tworzenie interfejsu, wygodną komunikację z bazą danych oraz możliwość uruchomienia całego systemu w kontenerach Docker.

Zastosowany zestaw technologii nie wymaga ręcznej instalacji wszystkich usług na komputerze użytkownika. Projekt został przygotowany tak, aby można było uruchomić

środowisko za pomocą Docker Compose, co ułatwia testowanie, prezentację i dalszy rozwój aplikacji.



Rys. 10.1: Stos technologiczny i podstawowa architektura systemu WorkFlow

10.1. Warstwa prezentacji

Warstwa prezentacji odpowiada za interfejs użytkownika dostępny z poziomu przeglądarki internetowej. Jej zadaniem jest wyświetlanie danych, obsługa formularzy, komunikacja z backendem oraz zapewnienie użytkownikowi dostępu do modułów zgodnych z jego rolą.

- **Next.js** – framework oparty na React, wykorzystany do budowy aplikacji frontendowej. Zapewnia strukturę projektu, routing oraz mechanizmy potrzebne do tworzenia nowoczesnych aplikacji webowych.
- **React** – biblioteka służąca do budowy interfejsu użytkownika w oparciu o komponenty.

- **TypeScript** – język rozszerzający JavaScript o statyczne typowanie. Zastosowanie TypeScript ułatwia kontrolę struktur danych przekazywanych pomiędzy komponentami i akcjami aplikacji.
- **Tailwind CSS** – narzędzie do stylowania interfejsu za pomocą klas narzędziowych. Umożliwia szybkie budowanie spójnych widoków bez konieczności tworzenia dużej liczby osobnych arkuszy CSS.
- **shadcn/ui** – zestaw komponentów interfejsu użytkownika, wykorzystywany jako baza dla formularzy, przycisków, pól wyboru, tabel i innych elementów aplikacji.
- **TanStack Query** – biblioteka wspierająca pobieranie i odświeżanie danych po stronie frontendu. Ułatwia obsługę stanów ładowania, błędów oraz ponownego pobierania danych.

W warstwie frontendowej istotne było również zachowanie spójności wyglądu w trybie jasnym i ciemnym. W tym celu interfejs został oparty na klasach i tokenach kolorystycznych, które pozwalają dostosować wygląd komponentów do aktywnego motywu.

10.2. Warstwa logiki biznesowej

Warstwa backendowa odpowiada za logikę biznesową systemu, obsługę API, autoryzację, walidację danych oraz komunikację z bazą danych i magazynem plików. Backend jest oddzielną usługą uruchamianą w kontenerze Docker.

- **NestJS** – framework backendowy dla Node.js, oparty na TypeScript. Umożliwia organizację kodu w moduły, kontrolery, serwisy i guardy, co ułatwia utrzymanie większej aplikacji.
- **Node.js** – środowisko uruchomieniowe JavaScript wykorzystywane przez backend.
- **JWT** – mechanizm tokenów wykorzystywany do autoryzacji użytkowników po zalogowaniu.
- **Guards w NestJS** – mechanizmy służące do ochrony wybranych endpointów, sprawdzania tokenu użytkownika, roli oraz tokenu technicznego dla urządzeń RCP.

- **DTO i walidacja danych** – struktury opisujące dane wejściowe API oraz walidujące poprawność żądań przed wykonaniem operacji biznesowej.

Backend udostępnia zarówno endpointy wykorzystywane przez frontend, jak i endpoint techniczny przeznaczony do odbierania zdarzeń z czytników RCP. Dzięki temu jedna warstwa logiki biznesowej obsługuje zarówno działania użytkowników, jak i zdarzenia pochodzące z urządzeń.

10.3. Warstwa danych

Dane systemu przechowywane są w relacyjnej bazie PostgreSQL. Baza danych zawiera tabele opisujące pracowników, zakłady, projekty, przypisania, umowy, certyfikaty, dokumenty, nieobecności, czas pracy, eksporty rozliczeniowe oraz wpisy audytowe.

- **PostgreSQL** – relacyjna baza danych wykorzystywana jako główne miejsce przechowywania danych systemu.
- **Prisma ORM** – narzędzie pośredniczące pomiędzy backendem a bazą danych. Prisma pozwala definiować modele danych w pliku `schema.prisma` oraz wykonywać operacje na bazie z poziomu kodu TypeScript.
- **Prisma Client** – wygenerowany klient wykorzystywany w kodzie backendu do komunikacji z bazą danych.
- **Prisma db push** – polecenie używane do synchronizacji struktury bazy danych ze schematem Prisma.

Zastosowanie Prisma pozwala utrzymać model danych blisko kodu aplikacji. Dzięki temu łatwiej kontrolować relacje między encjami i zachować zgodność pomiędzy typami wykorzystywanymi w kodzie a strukturą bazy danych.

10.4. Magazyn plików

System przechowuje dokumenty pracowników poza relacyjną bazą danych. Do tego celu wykorzystano MinIO, czyli magazyn obiektowy kompatybilny z API S3.

- **MinIO** – usługa odpowiedzialna za przechowywanie plików dokumentów.
- **Bucket dokumentów** – wydzielone miejsce w MinIO przeznaczone na pliki związane z pracownikami, umowami, certyfikatami i nieobecnościami.

- **Metadane dokumentów w PostgreSQL** – baza danych przechowuje informacje o nazwie pliku, właścicielu dokumentu i ścieżce do pliku, natomiast sam plik znajduje się w magazynie obiektowym.

Oddzielenie plików od danych relacyjnych upraszcza model bazy danych i pozwala uniknąć przechowywania dużych plików bezpośrednio w tabelach.

10.5. Warstwa infrastruktury

Do uruchomienia systemu wykorzystano Docker oraz Docker Compose. Każda główna część aplikacji działa jako osobna usługa, dzięki czemu można jasno określić odpowiedzialność poszczególnych komponentów.

- **Docker** – technologia konteneryzacji umożliwiająca uruchamianie usług w izolowanych kontenerach.
- **Docker Compose** – narzędzie służące do definiowania i uruchamiania wielu kontenerów jako jednego środowiska.
- **Nginx** – reverse proxy kierujące ruch użytkownika do frontendu oraz zapytania API do backendu.
- **Plik .env** – plik zawierający zmienne środowiskowe, takie jak hasła, tokeny, porty i adresy usług.
- **Plik .env.example** – przykładowy plik konfiguracyjny przechowywany w repozytorium bez rzeczywistych sekretów.

W środowisku Docker Compose uruchamiane są następujące usługi:

- frontend,
- backend,
- PostgreSQL,
- MinIO,
- Nginx.

Nginx upraszcza dostęp do aplikacji, ponieważ użytkownik korzysta z jednego adresu. Ruch do interfejsu użytkownika trafia do frontendu, natomiast ścieżki API są przekazywane do backendu.

10.6. Seedowanie danych

W projekcie przygotowano mechanizmy seedowania bazy danych. Ich zadaniem jest ułatwienie uruchomienia systemu oraz przygotowanie środowiska demonstracyjnego.

- **Seed minimalny** – tworzy podstawowe dane potrzebne do rozpoczęcia pracy, między innymi główny zakład oraz konta startowe dla ról administratora, HR, księgowości i brygadzysty.
- **Seed demonstracyjny** – tworzy pełniejszy zestaw danych, obejmujący pracowników, projekty, czytniki RCP, umowy, certyfikaty, dokumenty, nieobecności, wpisy czasu pracy oraz przykładowy eksport rozliczeniowy.

Seed demonstracyjny jest szczególnie przydatny podczas prezentacji projektu, ponieważ pozwala pokazać działanie systemu bez ręcznego wprowadzania wszystkich danych. Ze względu na to, że może on czyścić istniejące dane, jego uruchomienie wymaga dodatkowego potwierdzenia przez zmienną środowiskową.

10.7. Komunikacja pomiędzy usługami

W środowisku kontenerowym poszczególne usługi komunikują się ze sobą za pomocą nazw usług zdefiniowanych w Docker Compose. Backend łączy się z bazą PostgreSQL, komunikuje się z MinIO oraz udostępnia API dla frontendu i czytników RCP.

Frontend komunikuje się z backendem przez adres API skonfigurowany w zmiennych środowiskowych. W środowisku uruchamianym przez Nginx zapytania API mogą być kierowane przez ścieżkę `/api`, co pozwala zachować jeden zewnętrzny adres aplikacji.

10.8. Uzasadnienie wyboru technologii

Wybrane technologie pozwalają utrzymać spójny stos oparty głównie na TypeScript. Frontend i backend korzystają z tego samego języka, co ułatwia pracę nad typami danych i strukturą kodu. PostgreSQL zapewnia stabilną bazę relacyjną, Prisma upraszcza komunikację z bazą, a MinIO pozwala oddzielić dokumenty od danych relacyjnych.

Docker Compose umożliwia uruchomienie kompletnego środowiska za pomocą jednej konfiguracji. Jest to istotne zarówno podczas lokalnego testowania, jak i podczas przygotowania projektu do prezentacji lub wdrożenia na serwerze.

Zastosowany stos technologiczny jest wystarczający do obsługi aktualnego zakresu systemu i jednocześnie pozostawia możliwość dalszej rozbudowy, na przykład o powiadomienia, dodatkowe raporty, integrację z systemem księgowym lub zewnętrznym dostawcą tożsamości.

Bibliografia

- [1] SAP SE. *Na czym polega transformacja cyfrowa?* 2023. <https://www.sap.com/poland/resources/what-is-digital-transformation> (dostęp 21.04.2026).
- [2] Tomasz Małkus, Małgorzata Dziedzic. *System ERP*. 2025. https://mfiles.pl/pl/index.php/System_ERP (dostęp 21.04.2026).
- [3] Instytut Doskonalenia Produkcji. *BPMN w praktyce, czyli jak czytać i tworzyć modele procesów biznesowych, które naprawdę działają!* <https://www.doskonalenieprodukcji.pl/blog/bpmn-w-praktyce-czyli-jak-czytac-i-tworzyc-modele-procesow-biznesowych-ktore-naprawde-dzialaja> (dostęp 21.04.2026).
- [4] Robert C. Martin. *Czysta architektura. Struktura i design oprogramowania. Przewodnik dla profesjonalistów*. Helion, 2018.
- [5] Witek Pruchnicki. *Jak Pisać Czysty Kod?* 2024. <https://witekpruchnicki.com/jak-pisac-czysty-kod> (dostęp 21.04.2026).
- [6] SowaProgramuje. *Nazewnictwo w programowaniu – 20 cennych wskazówek*. 2017. <https://sowaprogramuje.pl/nazewnictwo-w-programowaniu-20-cennych-wskazowek/> (dostęp 21.04.2026).
- [7] Hanna Hrytsiuk. *Porównanie PRINCE2 i PMBOK*. 2023. https://mfiles.pl/pl/index.php/Por%C3%B3wnanie_PRINCE2_i_PMBOK (dostęp 21.04.2026).
- [8] Dropbox. *Czym jest single source of truth (SSOT)?* 2025. <https://www.dropbox.com/pl/resources/source-of-truth> (dostęp 21.04.2026).
- [9] Michał Wiatr. *COBIT i jego wykorzystanie dla zwiększania efektywności działania funkcji IT*. 2017. <https://itwiz.pl/cobit-jego-wykorzystanie-dla-zwiekszenia-efektywnosci-dzialania-funkcji/> (dostęp 21.04.2026).
- [10] ISO/IEC. *ISO/IEC 27001 - Information security management*. 2025. <https://www.iso.org/isoiec-27001-information-security.html> (dostęp 21.04.2026).
- [11] Ustawa z dnia 10 maja 2018 r. o ochronie danych osobowych. *RODO*. 2018. <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20180001000> (dostęp 21.04.2026).
- [12] resilia. *Co to jest norma ISO 27001 i dlaczego jest tak ważna dla organizacji?* 2024. <https://resilia.pl/blog/iso-27001-czym-jest-jakie-daje-korzysci/> (dostęp 21.04.2026).

-
- [13] Kapitan Hack. *Co to jest RBAC? Porównanie RBAC z ABAC*. 2023. <https://kapitanhack.pl/co-to-jest-rbac-porownanie-rbac-z-abac> (dostęp 21.04.2026).
 - [14] gigacloud. *Konteneryzacja i jej zalety*. 2024. <https://gigacloud.eu/pl/articles/containerization-and-its-benefits/> (dostęp 21.04.2026).
 - [15] Michał Wolski. *Struktura dokumentów wymagań – część 1*. 2015. <https://wolski.pro/2015/04/struktura-dokumentw-wymagan-cz-1/> (dostęp 21.04.2026).
 - [16] Refactoring.Guru. *Adapter*. <https://refactoring.guru/pl/design-patterns/adapter> (dostęp 21.04.2026).

Spis rysunków

5.1. Ogólny diagram przypadków użycia systemu WorkFlow	34
5.2. Diagram przypadków użycia dla modułu kadrowego	35
5.3. Diagram przypadków użycia dla rejestracji czasu pracy i rozliczeń . . .	36
5.4. Diagram przypadków użycia dla administracji i zarządzania strukturą organizacyjną	37
7.1. Widok desktopowy – pulpit główny systemu WorkFlow	50
7.2. Widok mobilny – pulpit główny systemu WorkFlow	51
7.3. Widok desktopowy – profil pracownika	52
7.4. Widok mobilny – profil pracownika	53
8.1. Logiczny model danych systemu WorkFlow	56
10.1. Stos technologiczny i podstawowa architektura systemu WorkFlow . . .	71

Spis tabel

8.1. Główne encje modelu danych systemu WorkFlow	59
8.2. Struktura tabeli TimeEntry	61

Spis listingów

1. Fragment modelu TimeEntry w schemacie Prisma 63
2. Synchronizacja bazy danych ze schematem Prisma 64